Hadoop & Spark - ISEN 2025/2026

Chapter 3: How to process data with Spark

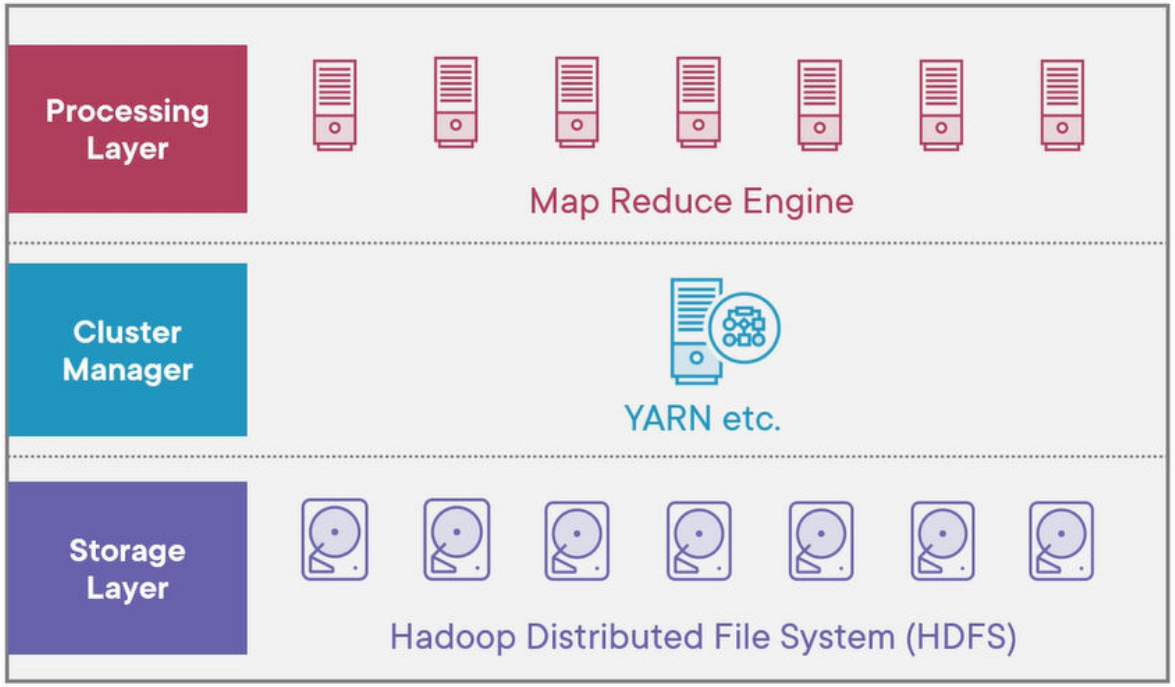
Goals of this chapter

In this chapter we will see:

- What is Spark and how it came to be
- Spark notions such as RDDs, datasets, UDFs ...

How did Spark come to be? (1/3)

As a reminder for Hadoop



How did Spark come to be? (2/3)

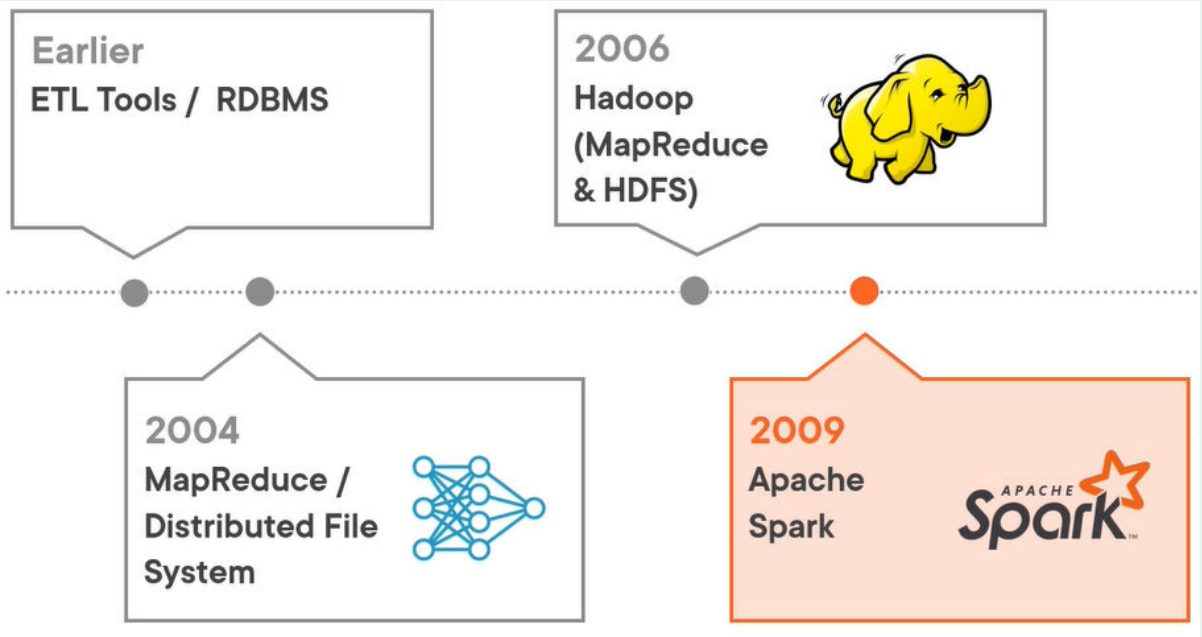
Back to a single cohesive stack



How did Spark come to be? (3/3)

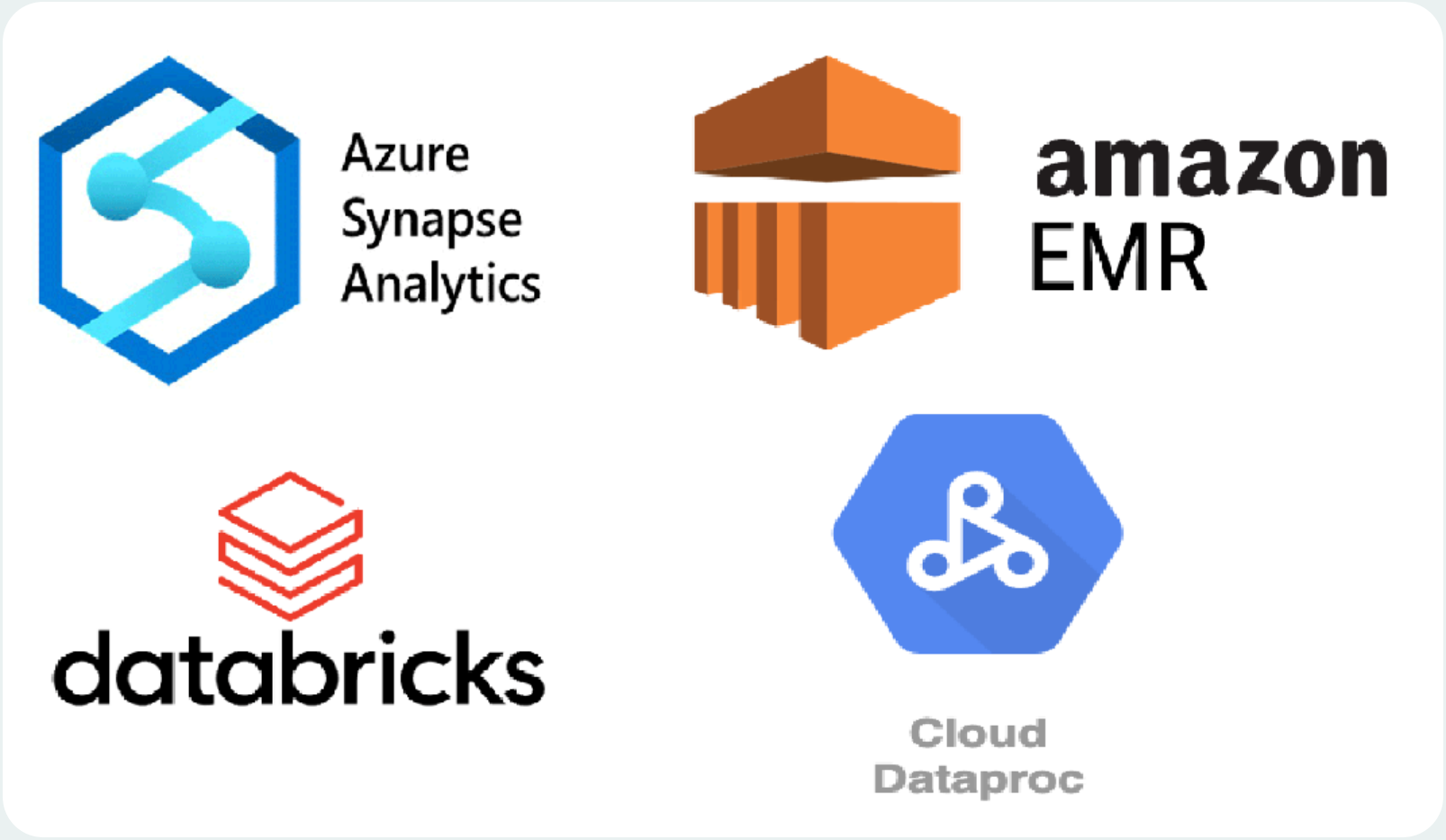
New features from previous technologies:

- Work with structured **and** unstructured data
- **In-memory engine** for distributed data processing (based on Java).
- Multiple language support (ex: Python, SQL, R, Java & Scala)
- Batch & Streaming

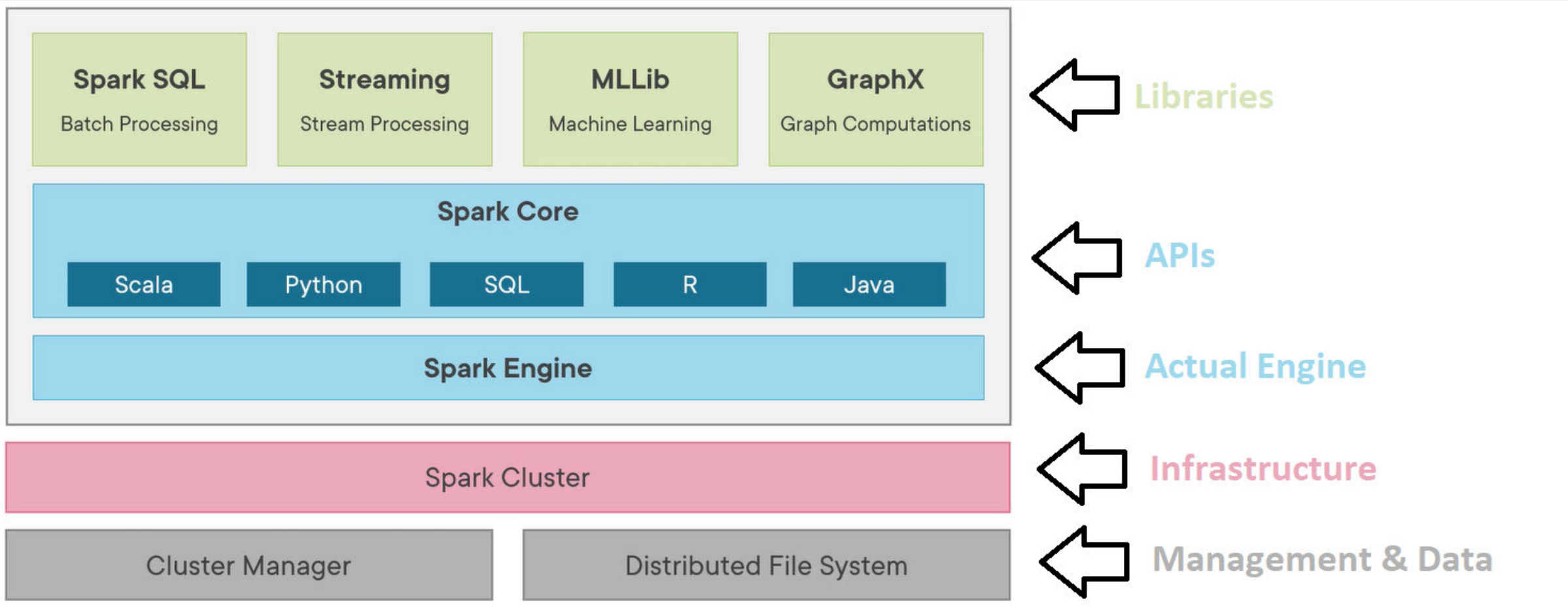


Today Spark is used locally or in the Cloud

Example of services using Spark (directly or behind the scenes):



Spark Structure (1/2)



Spark Structure (2/2)

The structure is always the same, but the components may change in this open-source stack.
Here are some examples:

- **Cluster Manager**: YARN, Kubernetes
- **Distributed File System**: HDFS, Azure Blob Storage, Amazon S3 buckets, Google Cloud Storage
- **Other Storage**: relational databases, NoSQL
- **Core**: Scala, Python, SQL, R
- **Visualization**: Power BI, Tableau
- **Streaming**: Kafka, Event Hubs

More examples on Apache Spark third-party projects

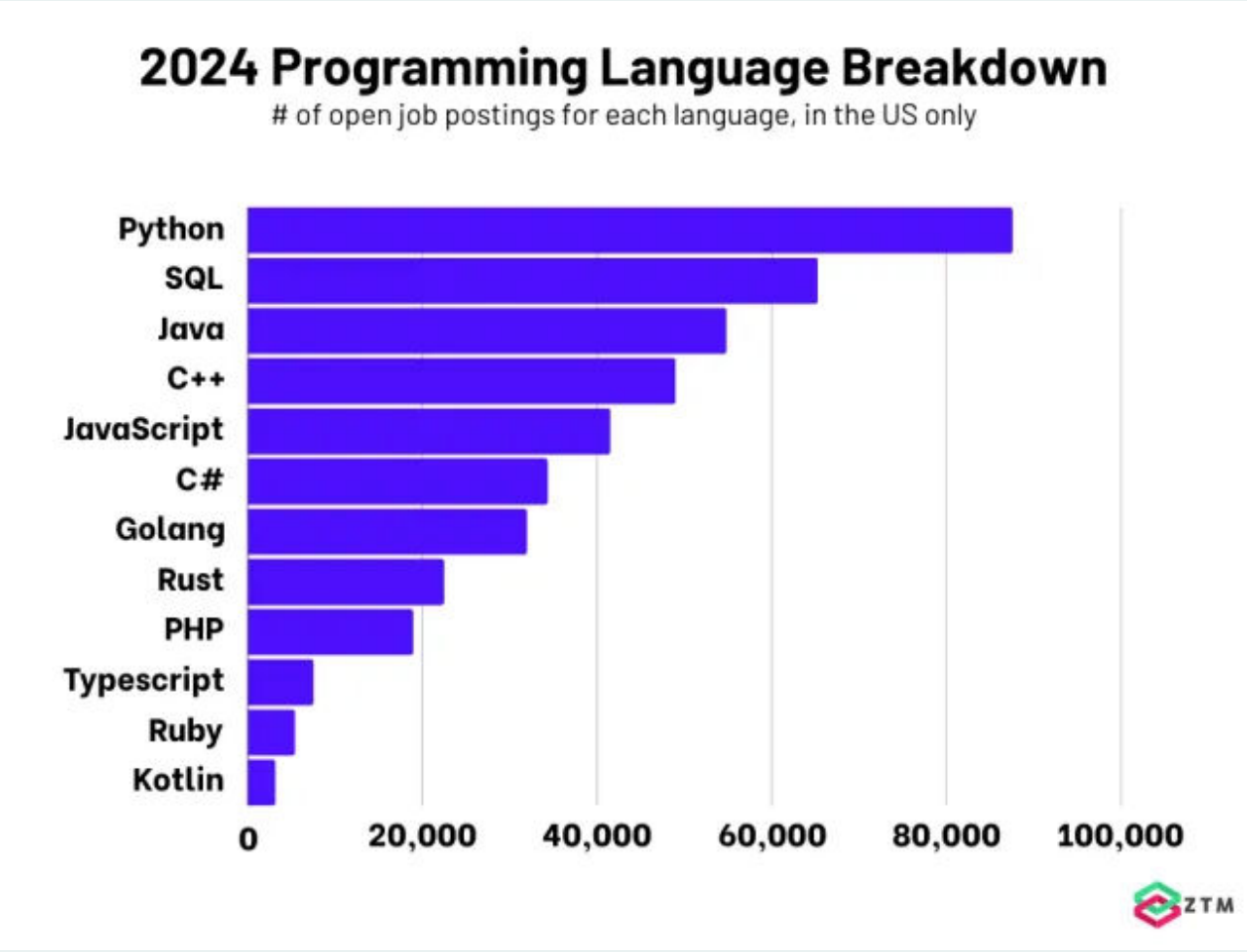
Which language to choose ?

As stated before, Spark supports multiple programming languages for developing applications.

However the two most used in real use cases are currently using either:

- **Scala** (the native language of Spark)
- **Python** (PySpark benefits from python huge popularity).

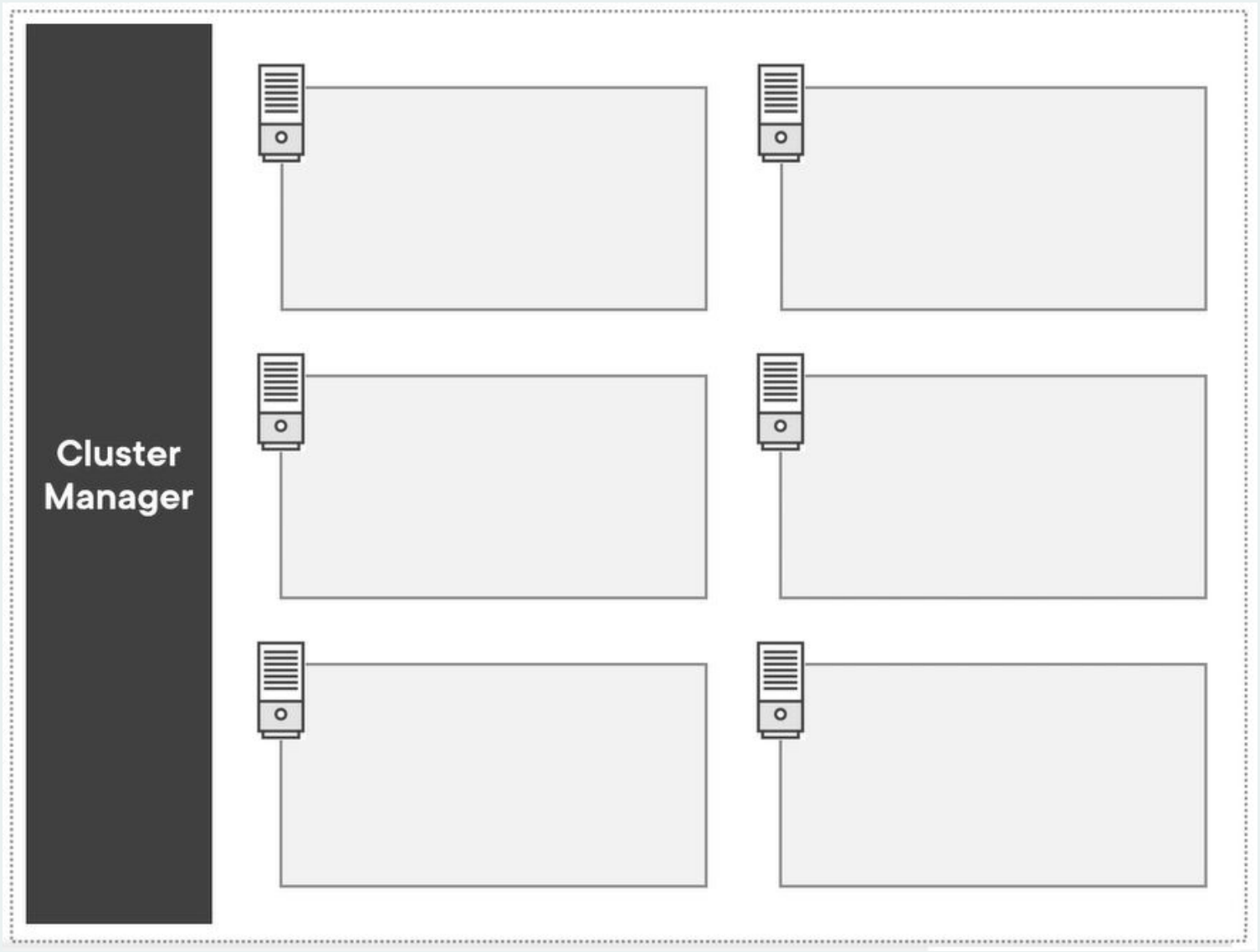
In the following course we will use **PySpark**, but the only reason for this choice is that it is more convenient to use with an online Notebook.



Spark execution: overview (1/4)

Let's see how Spark executes the requests.

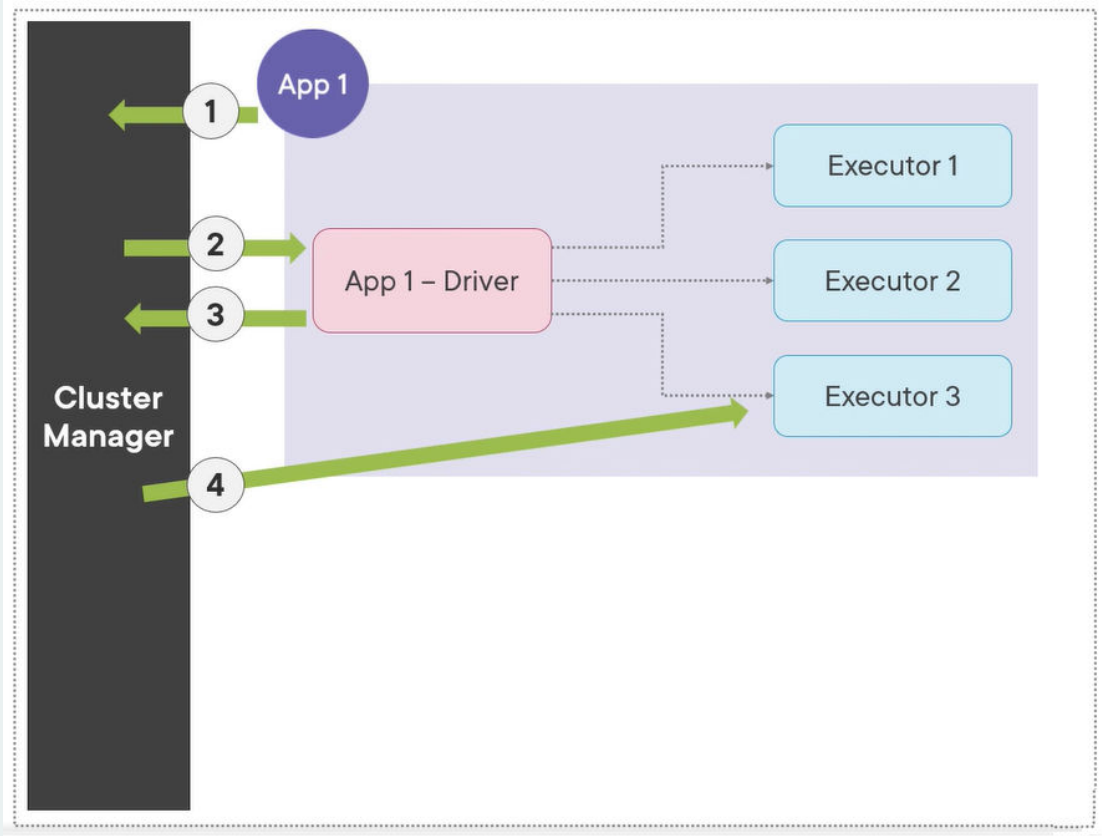
This is our cluster (with nodes + cluster manager)



Spark execution: overview (2/4)

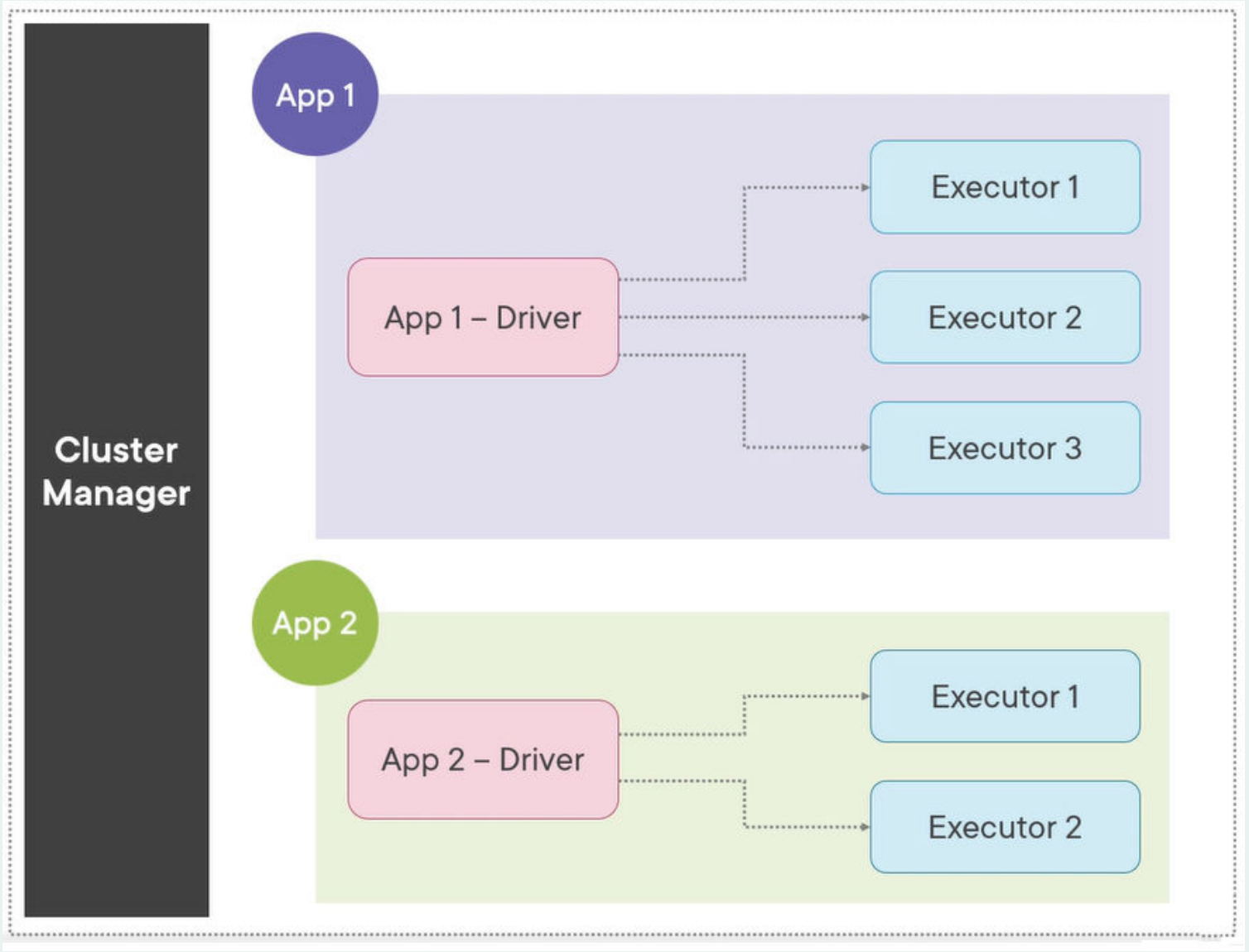
Imagine you launch an application named `App 1`.

- **1**: A user creates and deploys the `App 1`
- **2**: `Cluster Manager` Launch a `Driver Process`
- **3 & 4**: The `Driver Process` asks `Cluster Manager` where is the data located and then creates the `Executor Processes`



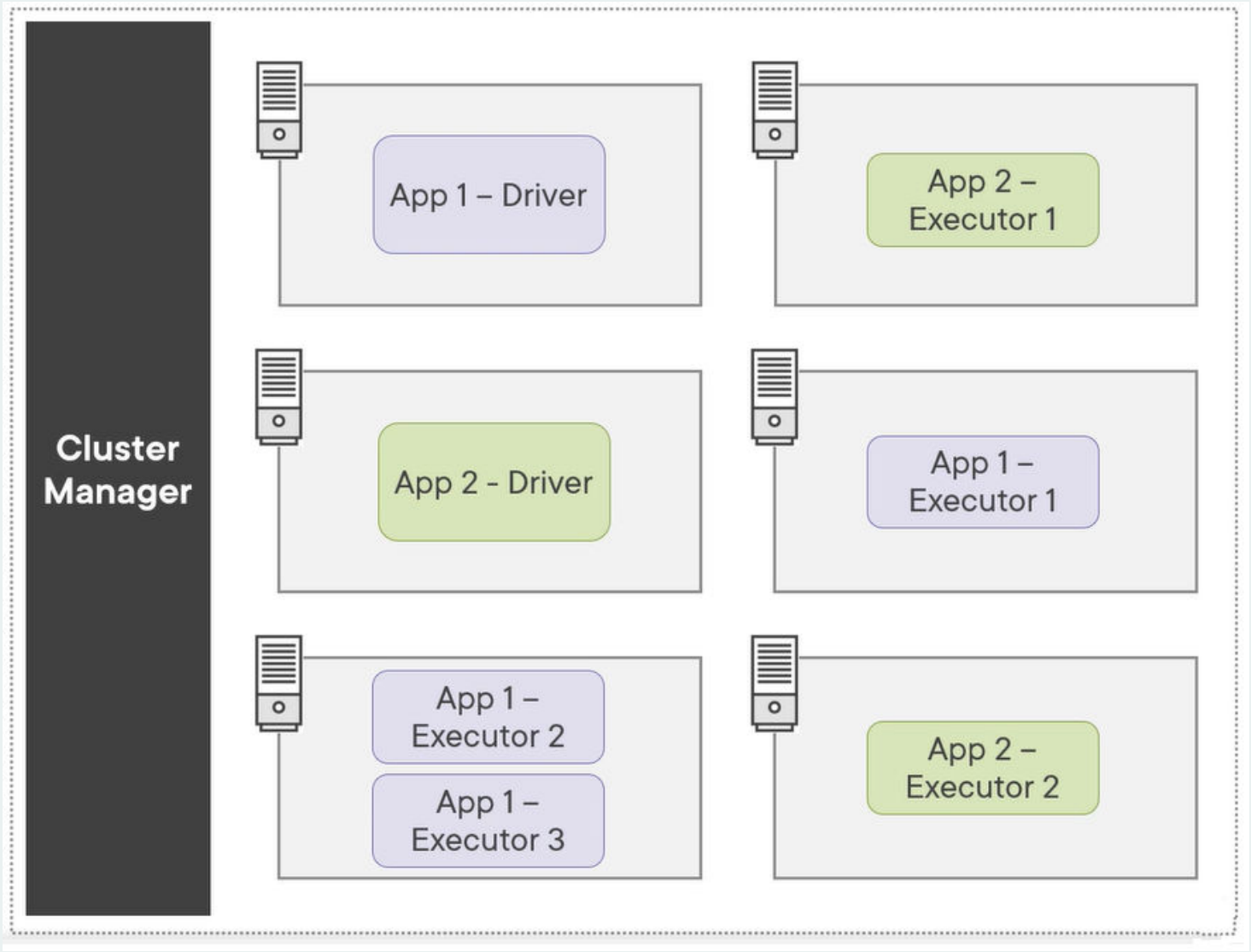
Spark execution: overview (3/4)

If another application is launched in parallel, it follows the same pattern



Spark execution: overview (4/4)

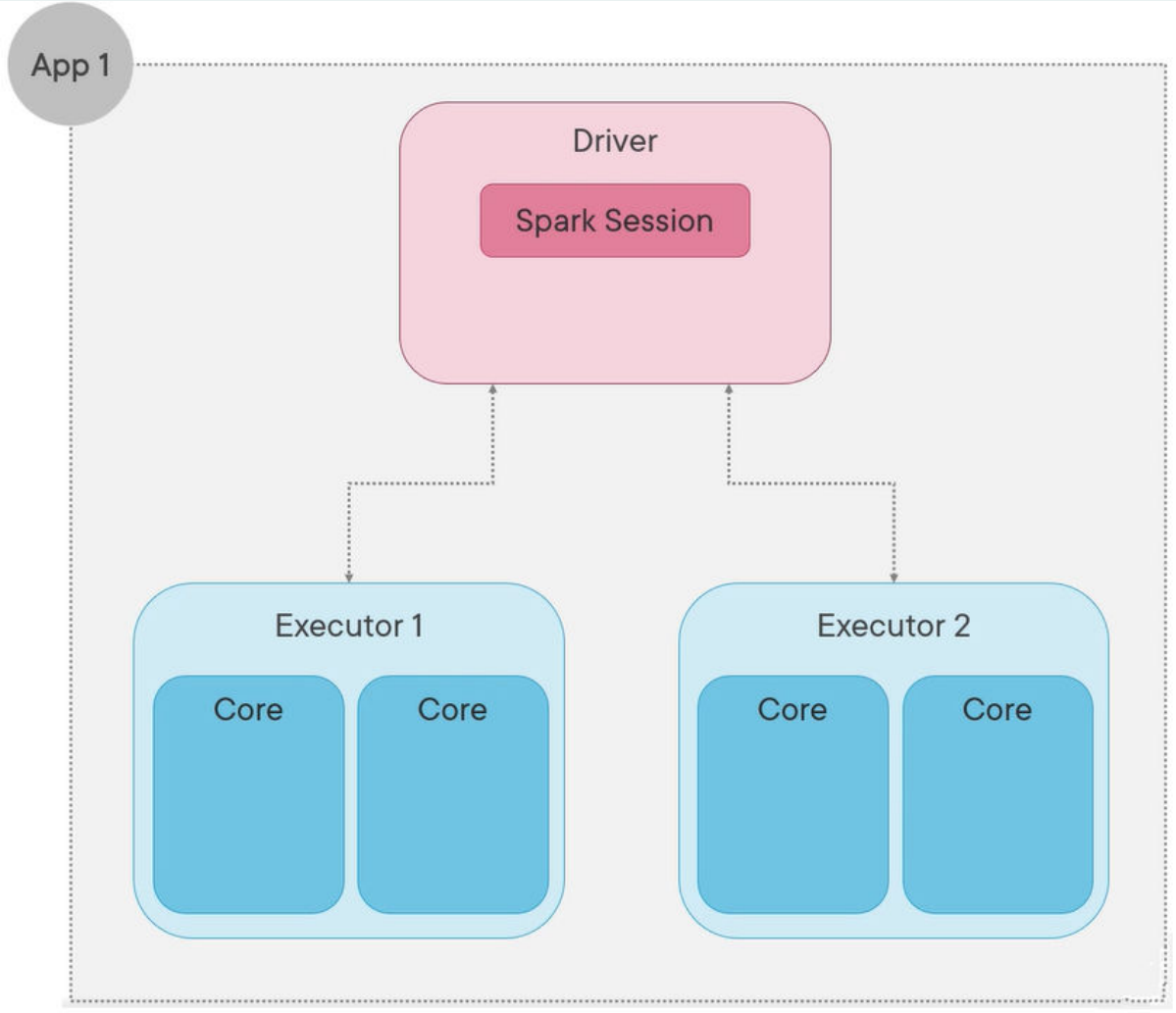
If we get back to our cluster, each Node may host one or several processes, and each application may or may not be split across multiple machines.



Spark execution: zoom (1/4)

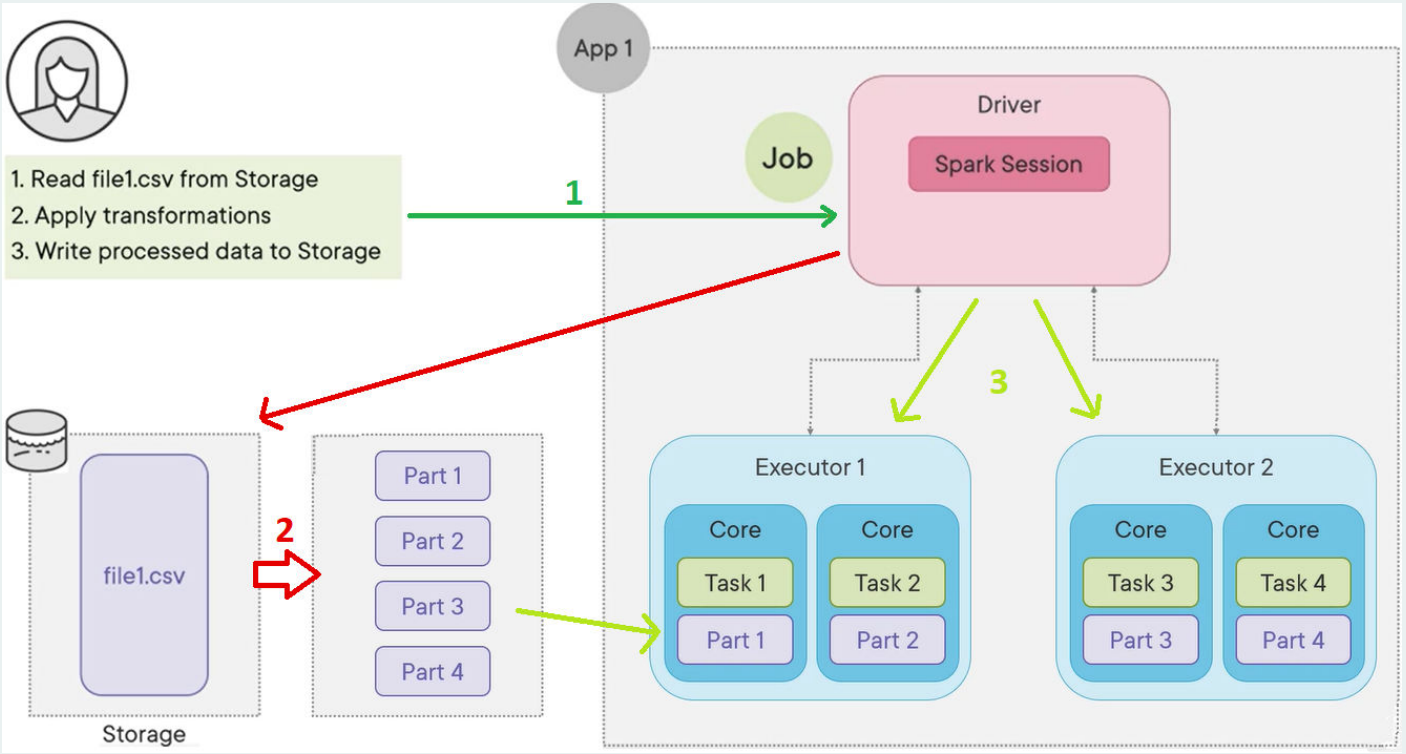
If we zoom in on the Drivers and executors we have:

- The `Driver` analyzes the code from the user and splits the work across `Executors`.
- Each Node is a physical or virtual machine with a given number of cores, RAM, storage ...



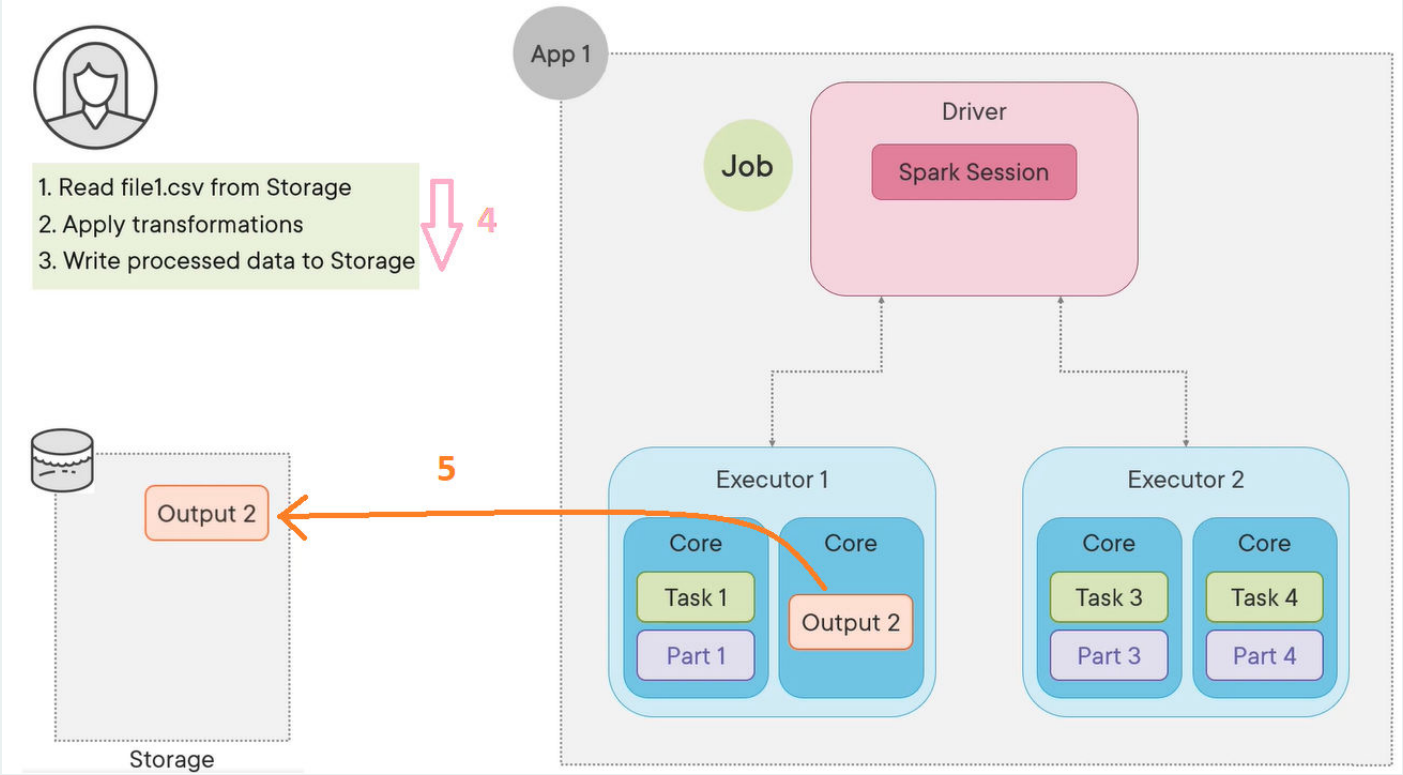
Spark execution: zoom (2/4)

- **1** The `Job` is sent to the `Driver`
- **2**: The `Driver` splits the data into smaller partitions (not always the same size)
- **3**: The `Driver` splits the `Job` into `Tasks` (1 Task = 1 Partition) and sends them to the executors as uniformly as possible



Spark execution: zoom (3/4)

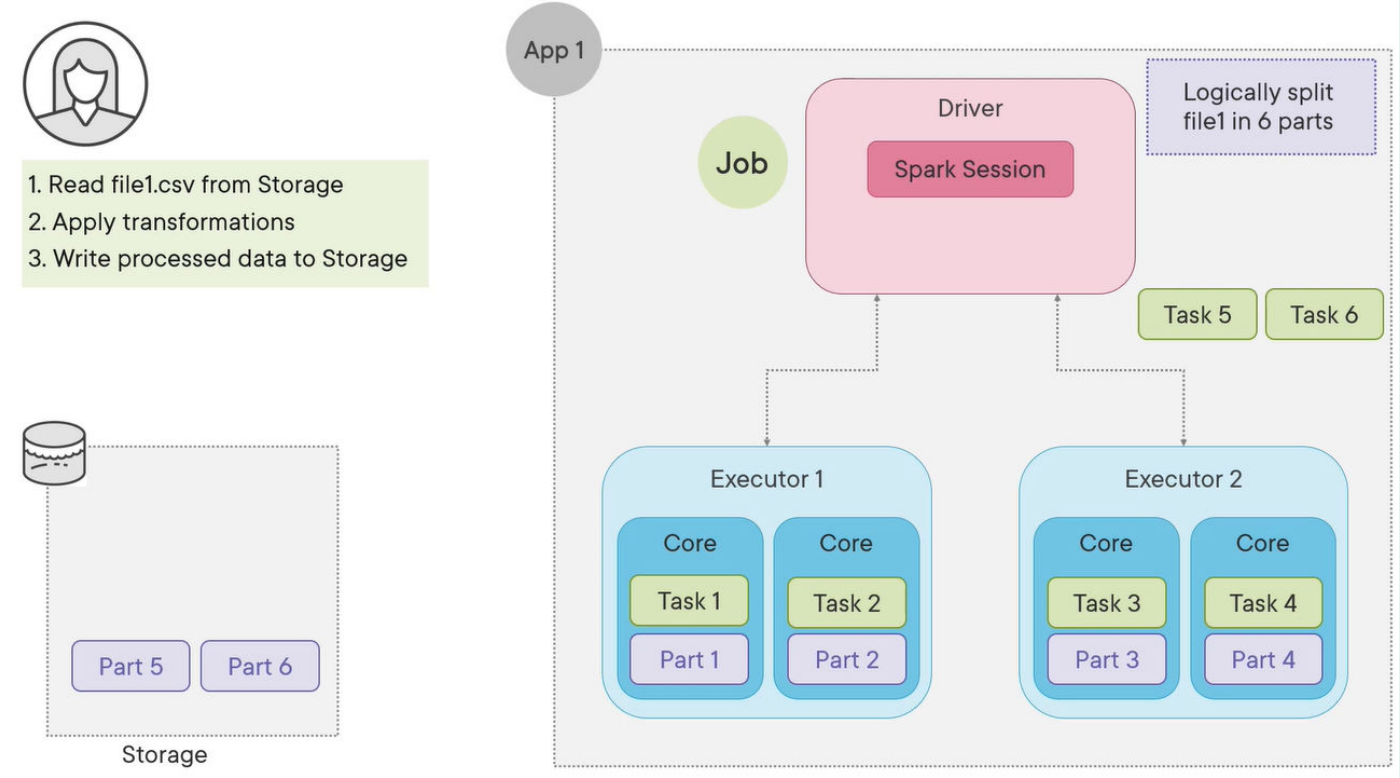
- **4** The `Job` is applied on each node individually
- **5**: Once a Node has finished its work, it will save the output in the Storage. This means the output is saved in multiple files, one for each `Task`



Spark execution: zoom (4/4)

- If there are more `Tasks` than available Nodes, `Tasks` may be queued up, waiting for another process to finish.

In order to process the jobs faster you may increase the number of cores (vertical scaling) or increase the number of machines (horizontal scaling).



Spark APIs

Historically, Spark has 3 main APIs to handle Data jobs:

- **Resilient Distributed Datasets (RDDs)**
- **DataFrames**
- **Datasets**

RDD overview

- **This is the native representation of data in memory in Spark**:
  - All processing in Spark happens using RDDs
  - Native data structure of Spark Core Library
- Introduced at the creation of Spark
- Low-level API
- Spark does **NOT** apply any optimization to the code
- There is compile-time safety
- Supported in Python, Scala, Java, and R

DataFrame overview

- High-level Spark API
- Introduced in Spark 1.3
- Based on RDD but in tabular format
- There is **NO** compile-time safety
- Spark applies optimizations to the code behind the scene
- Supported in Python, Scala, Java, and R

Dataset overview

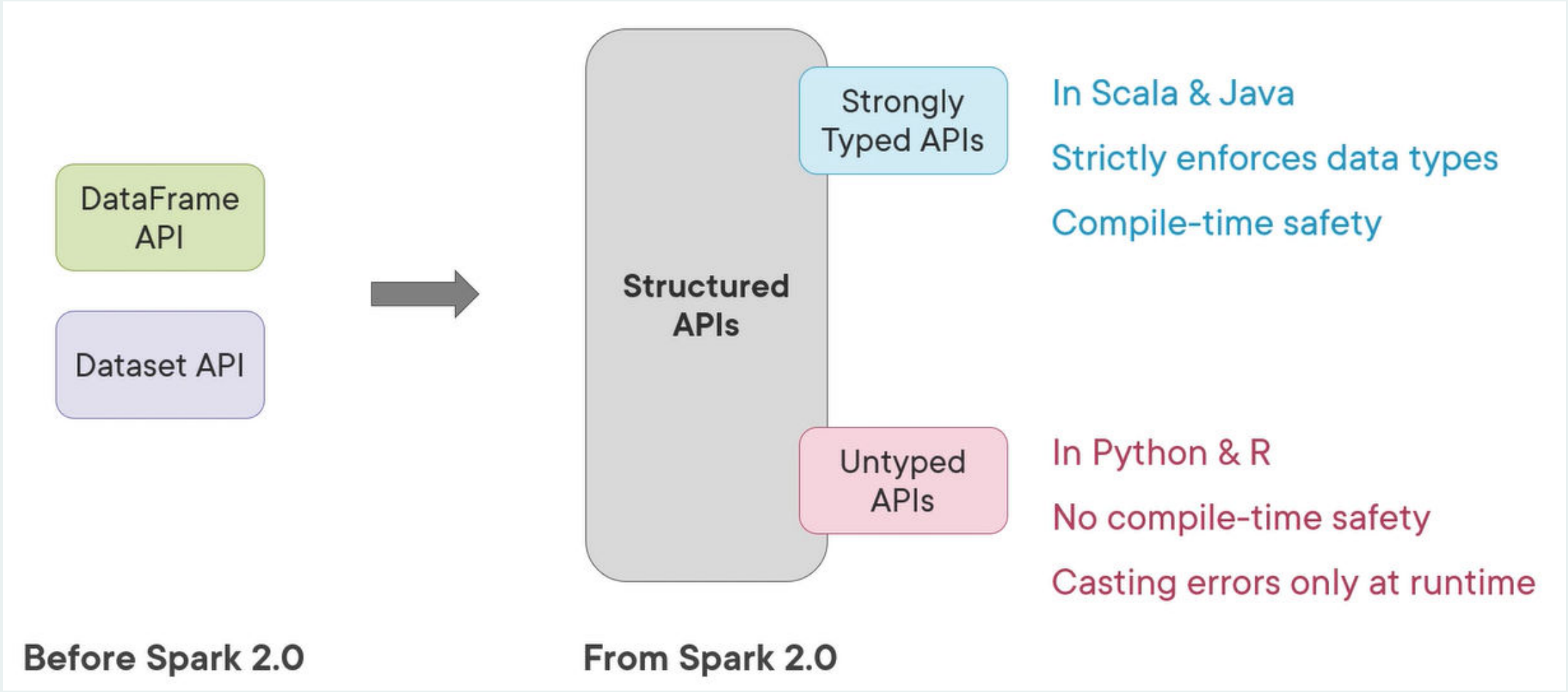
- High-level Spark API
- Introduced in Spark 1.6
- Based on DataFrames and RDDs but with type-safe and object-oriented capabilities
- There is compile-time safety
- Spark applies optimizations to the code behind the scene
- Supported **in Scala and Java only**

Common aspects of RDD, DF, and DS

- **Lazyness**: Spark does not apply changes right away, but waits for the first time you need to execute the code to launch the transformations



As of Spark 2.0



Details on RDDs

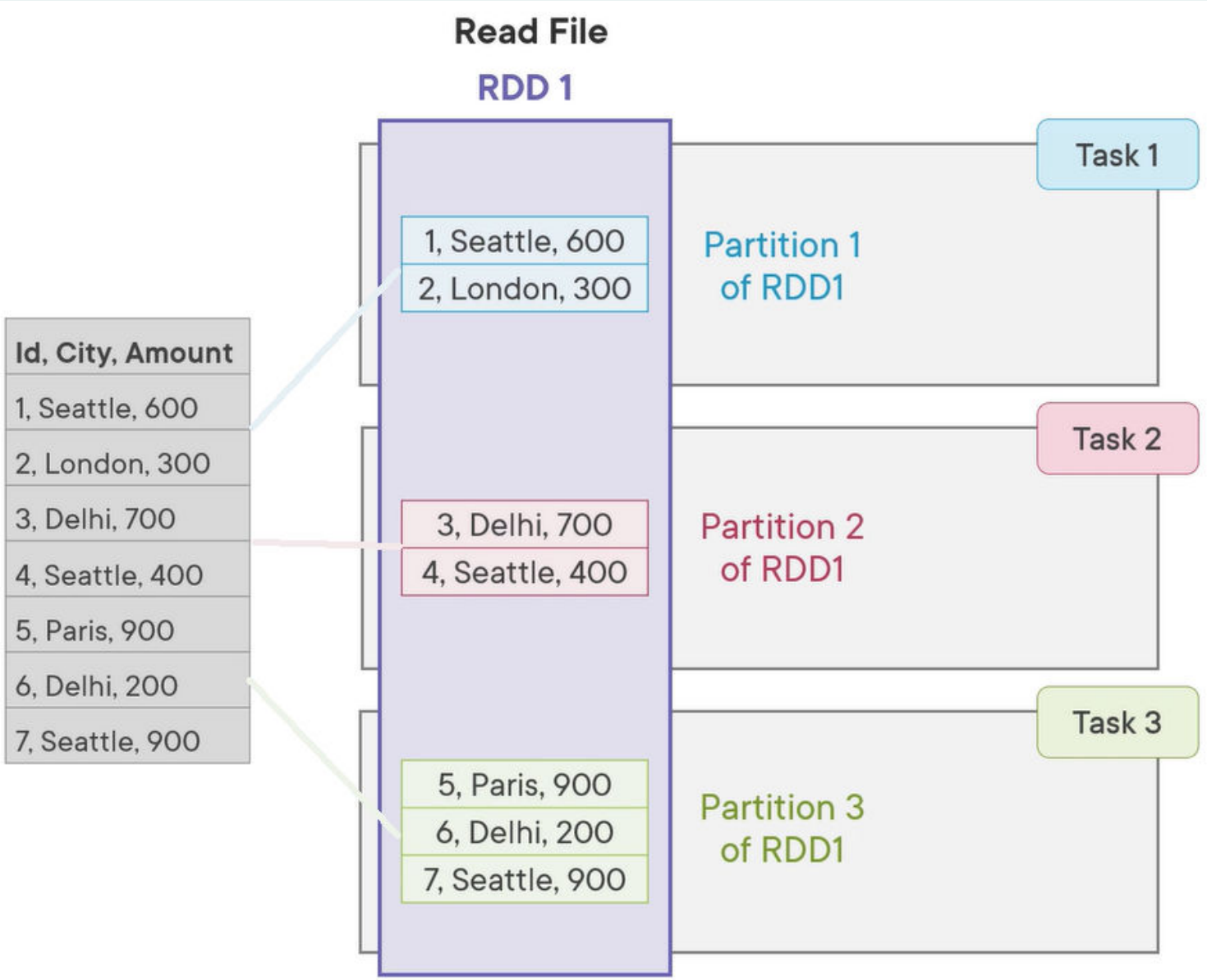
Let's take an in-depth look at RDDs, which is the native representation of data in Spark.

In-memory partitions

RDDs are **in-memory** and **partitioned** objects.

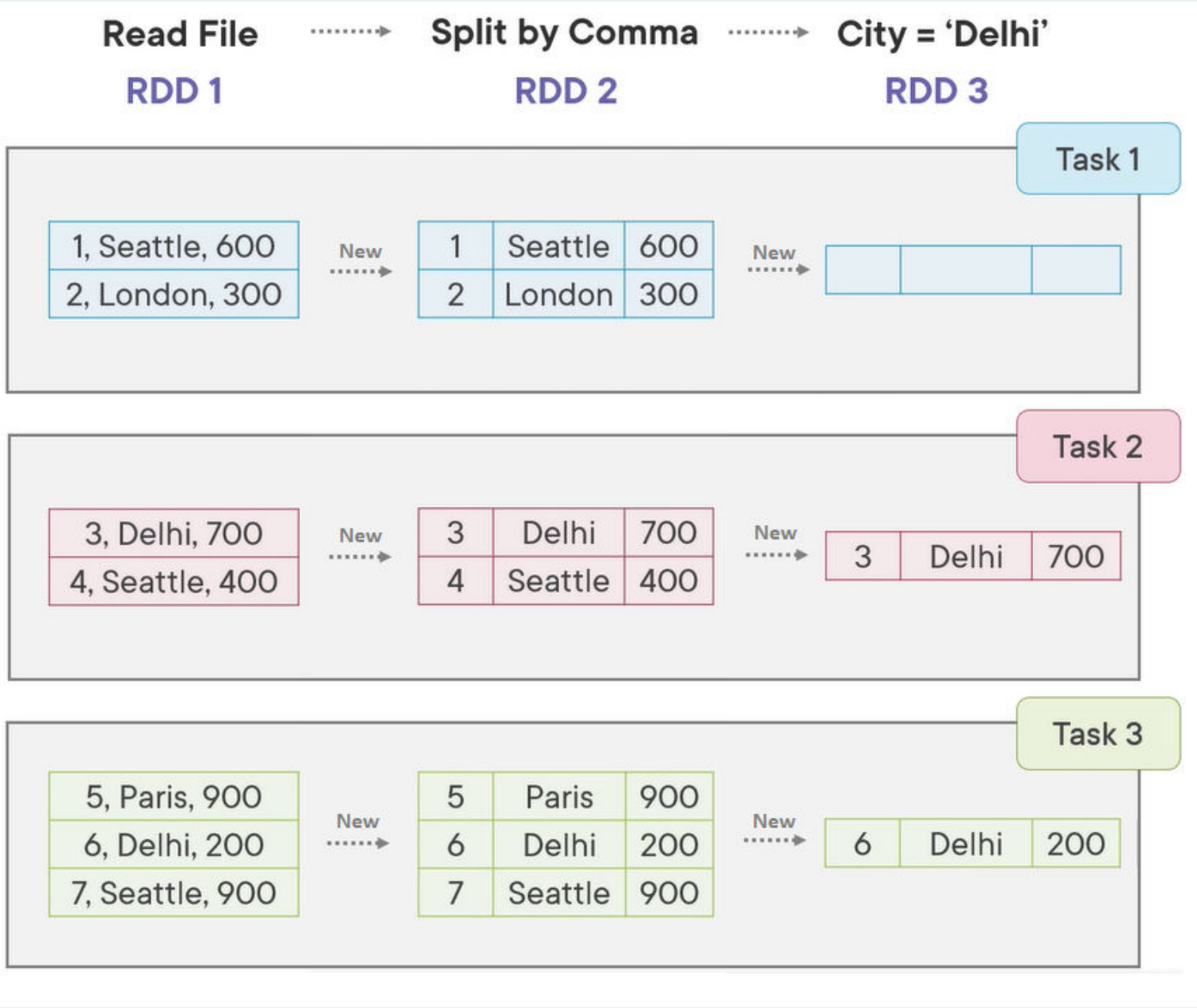
They **do not have a schema** and are just a distributed collection of elements.

Note: The number of partitions will define the number of tasks to apply a job to the data.



Read-only

A RDD is **read-only**, this means that every time an operation is made, **a new RDD** is created **(and may be empty)**



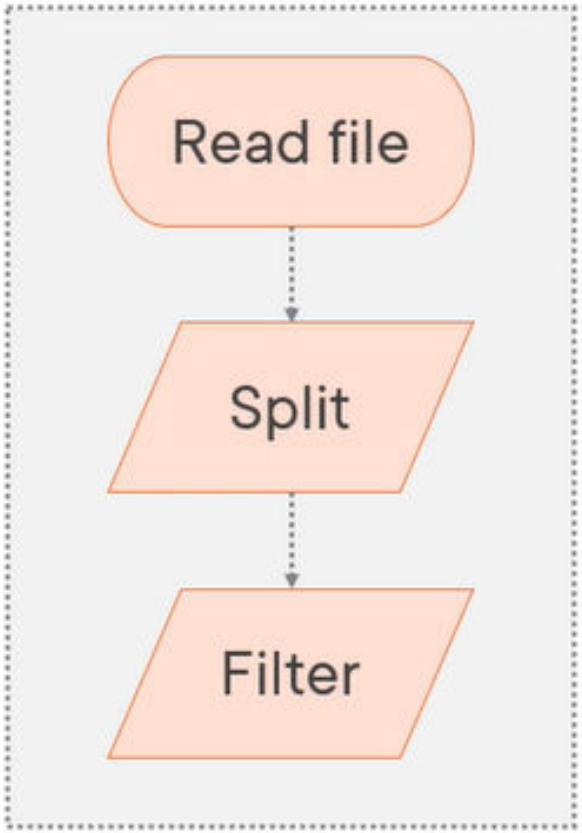
Lineage Graph

When a job is launched, a **Lineage Graph** is created.

It describes the steps to follow to create an RDD.

When a task is launched, it applies the graph to a given partition.

1. Read file1.csv from Storage
2. Split data by comma
3. Filter where City='Delhi'
4. Write processed data to storage

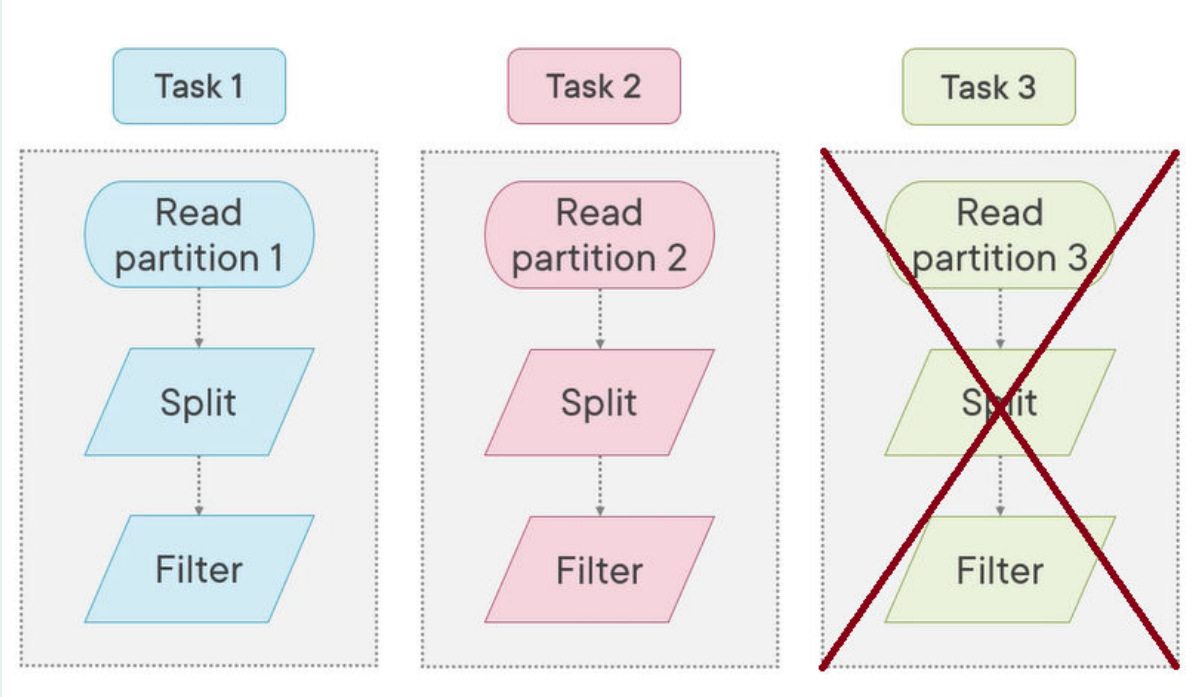


Resilience

As tasks are all described as a **Lineage Graph**, in case of a **failure or crash**, Spark can **re-execute** the failed tasks from the beginning.

It does not have to do the whole process again, **only the failed partition**.

> **Note:** The error needs to be transient (=that can be fixed by retrying)



Spark Lab

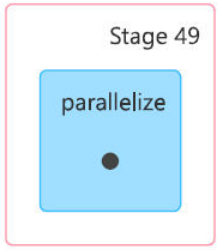
- Setup PySpark (3.1)
- RDDs (3.2)

A simple job in the UI

Details for Job 32

Status: SUCCEEDED
Submitted: 2023/08/29 12:50:51
Duration: 37 ms
Completed Stages: 1

▸ Event Timeline
▾ DAG Visualization



▾ Completed Stages (1)

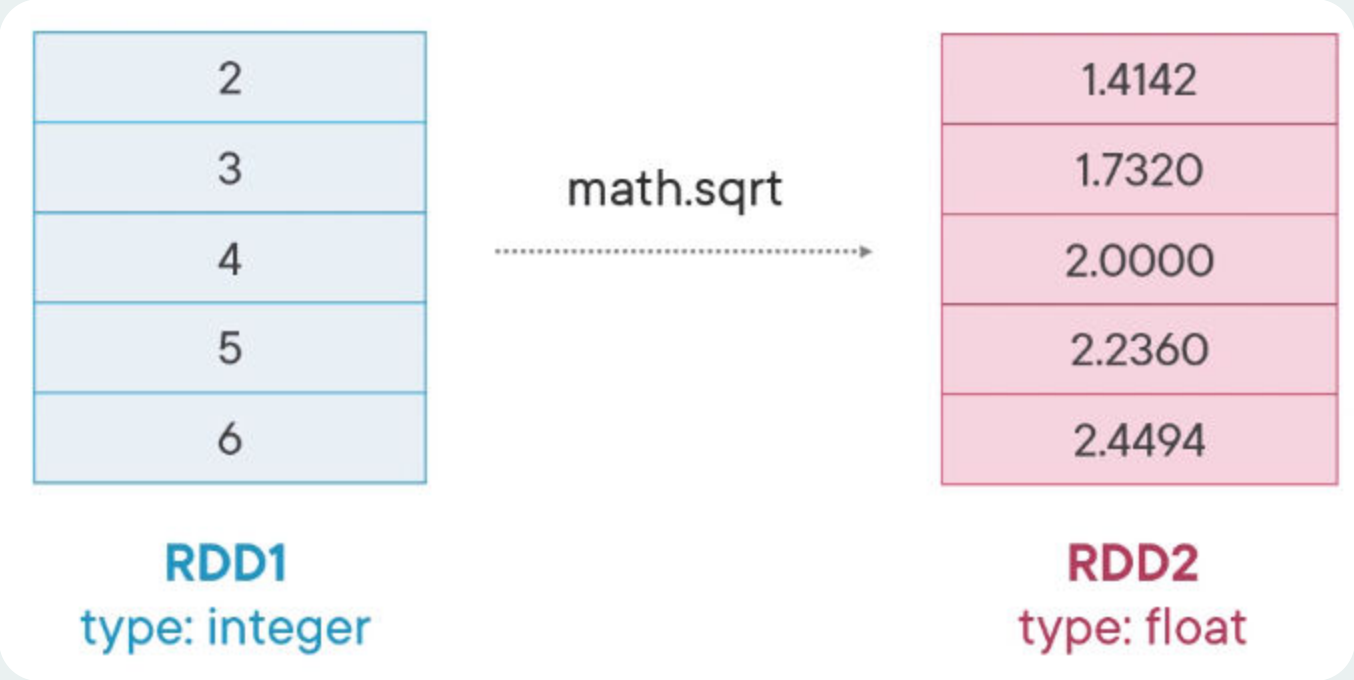
Page: 1 — 1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id ▾	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
49	collect at <ipython-input-48-07436937a221>:1 +details	2023/08/29 12:50:51	21 ms	2/2				

Page: 1 — 1 Pages. Jump to 1 . Show 100 items in a page. Go

Pair RDD (1/3)

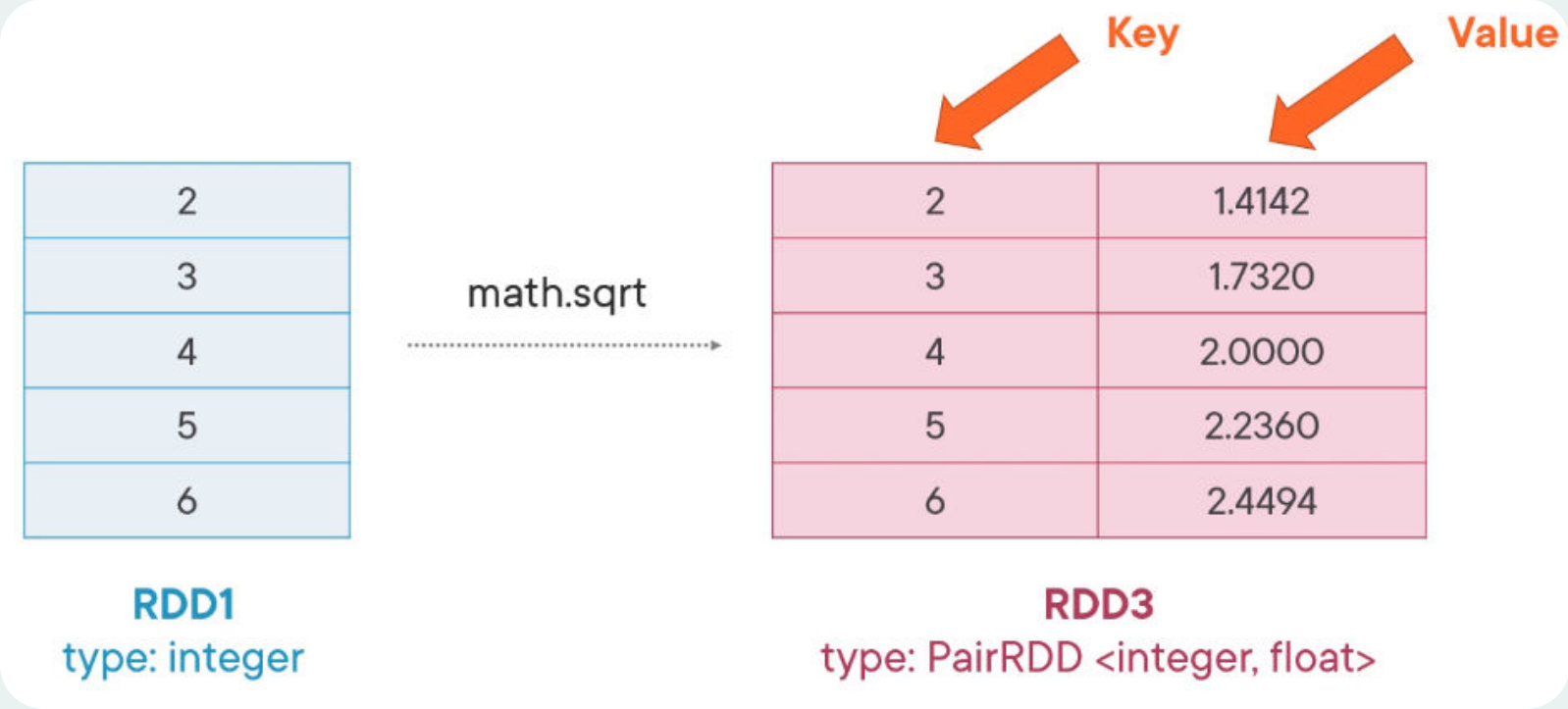
Sometimes the use of simple RDDs is not enough.
Here is an example, where we apply the square root to some numbers.



We have 2 RDDs as expected, but by just looking at the second RDD, you do not know what was the original number.

Pair RDD (2/3)

This is where `Pair RDDs` come in handy:



They are composed of 2 elements, a key (not necessarily unique) and an associated value.

Pair RDD (3/3)

Spark has useful special operations for `Pair RDDs`:

- reducebyKey
- sortByKey
- countByKey
- ...

Spark Lab

- Pair RDD (3.3)

Operations on RDDs

Two types of operations:

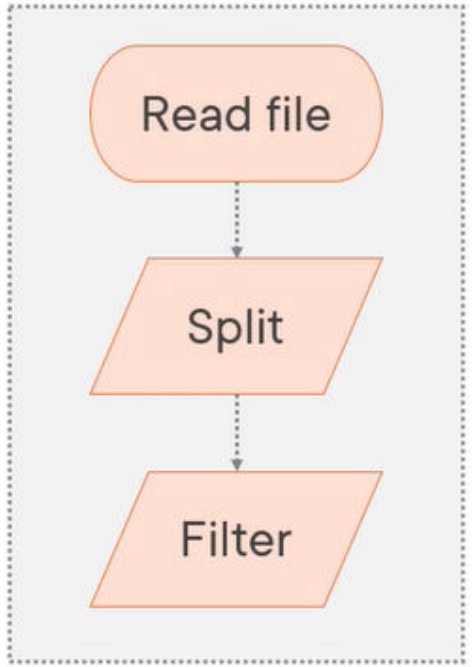
- **Transformations**
- **Actions**

  Let's examine both of those.

Transformation

- A transformation is a function that returns DataFrames, Datasets, or RDDs. **It represents the data**.
- Transformations are part of the `Lineage Graph`. For example: read a file, convert types, filters, group by, join...
- It often requires little computation.

1. Read file1.csv from Storage
2. Split data by comma
3. Filter where City='Delhi'
4. Write processed data to storage



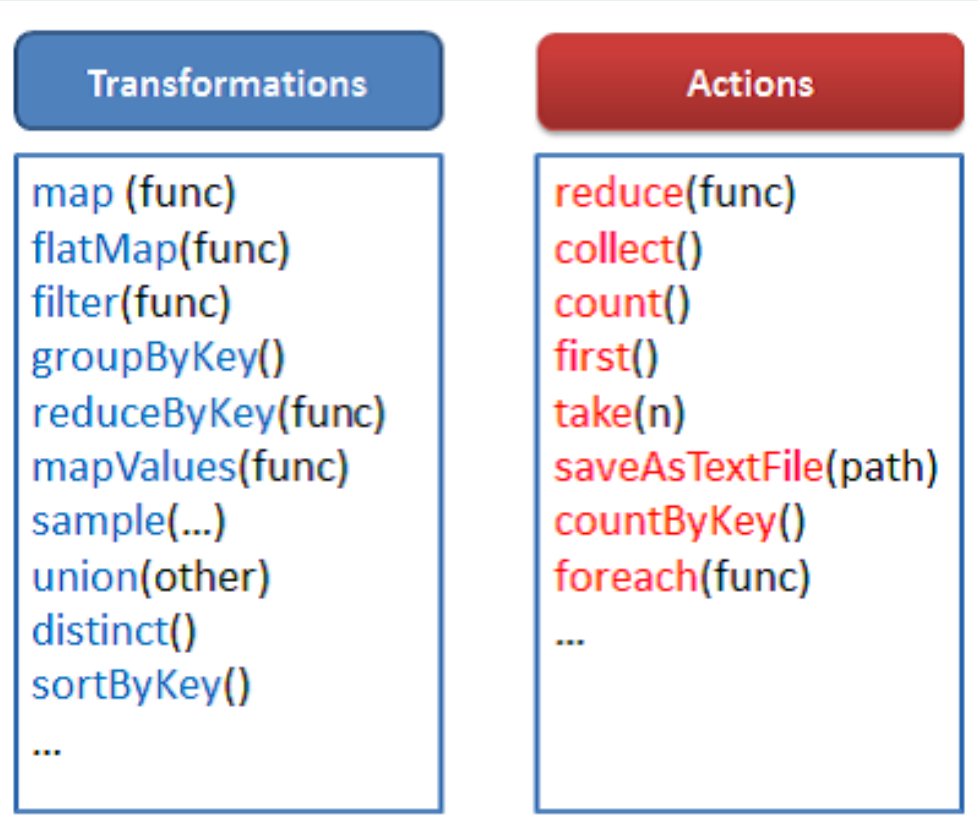
Actions

- An action is an operation that actually **loads the data**.
- It requires often a lot of computation as it not only represents the data but makes concrete operations as well.
- Actions are for example: reduce, count, iterations (loop), write, print...

Transformation VS Actions (1/2)

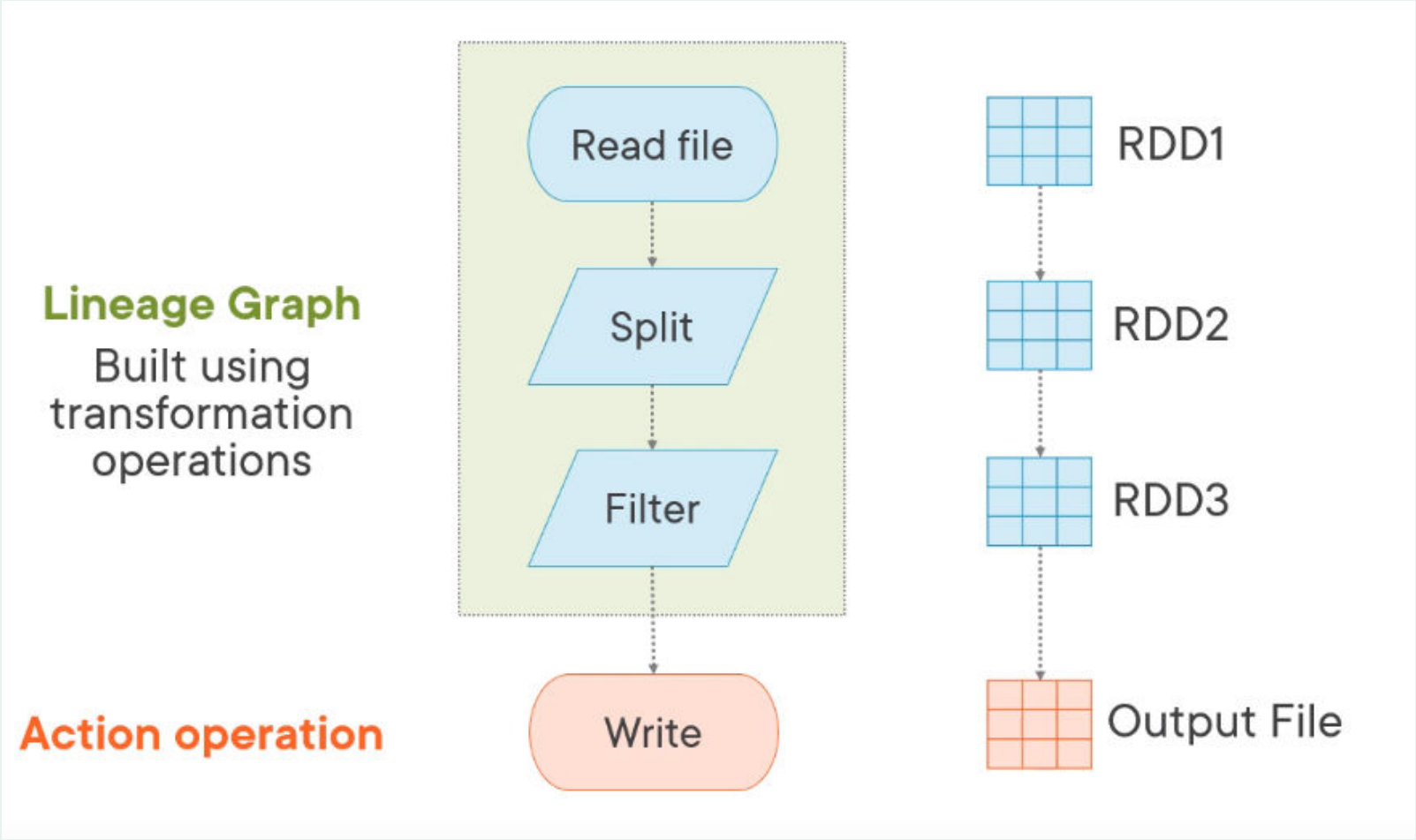
Very important: `Transformations` are always **`lazy operations`** which means Spark wait for `Actions` to execute them.

It is generally advised to avoid using actions whenever a transformation is possible to optimize the compute needed.



Transformation VS Actions (2/2)

An action always ends a Job



Spark Lab

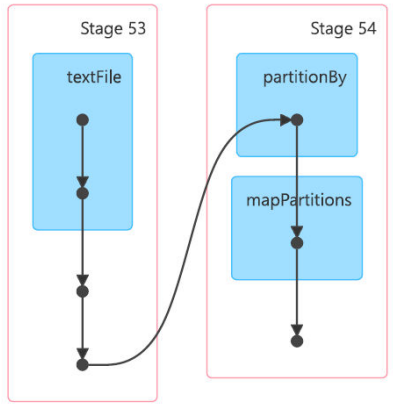
- Transformations & Actions (3.4)

An advanced job in the UI

Details for Job 36

Status: SUCCEEDED
Submitted: 2023/08/29 12:57:17
Duration: 0.8 s
Completed Stages: 2

▸ Event Timeline
▾ DAG Visualization



▾ Completed Stages (2)

Page: 1 — 1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id ▾	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
54	runJob at PythonRDD.scala:179 +details	2023/08/29 12:57:18	0.2 s	1/1			3.5 KiB	
53	reduceByKey at <ipython-input-51-81aa59527475>:3 +details	2023/08/29 12:57:17	0.5 s	2/2	2.3 MiB			6.4 KiB

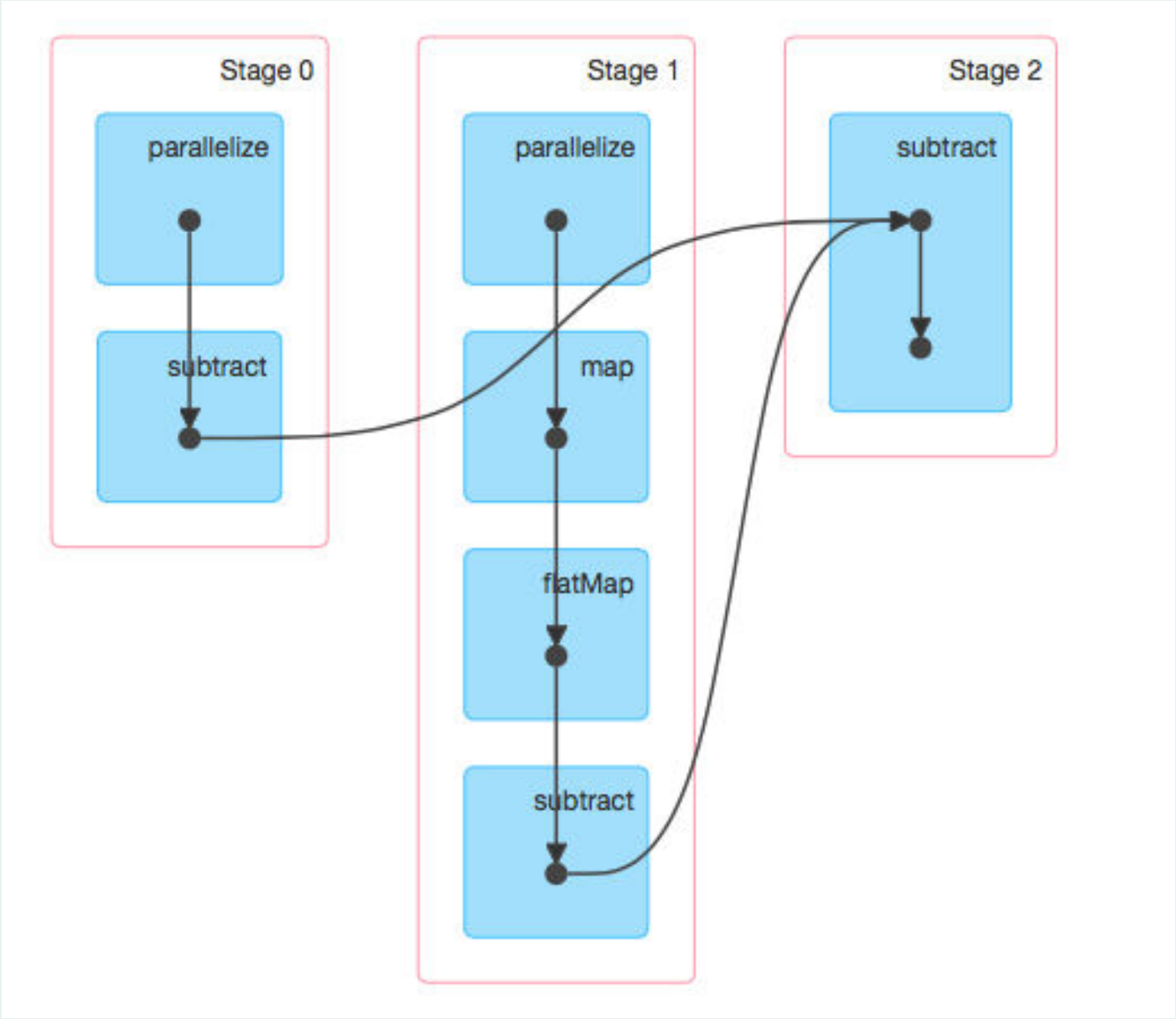
Page: 1 — 1 Pages. Jump to 1 . Show 100 items in a page. Go

DAGs

DAG or **Directed Acyclic Graph** is a representation of linked operations in Spark.

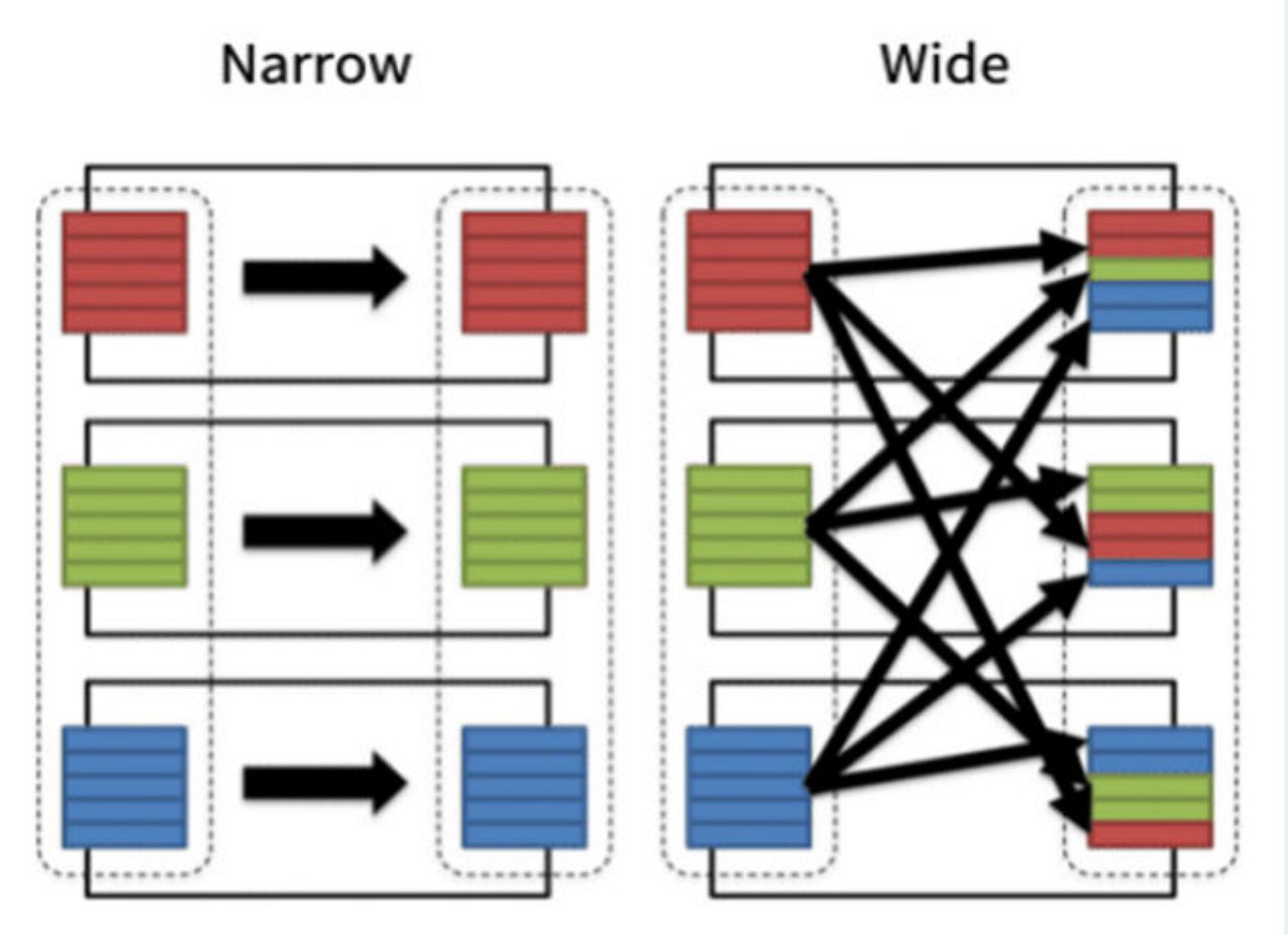
It is composed of `Stages`, a series of `Transformations` that can be performed on each of the RDD's partitions without having to access data in other partitions.

It always ends with an `Action`.



2 types of transformations

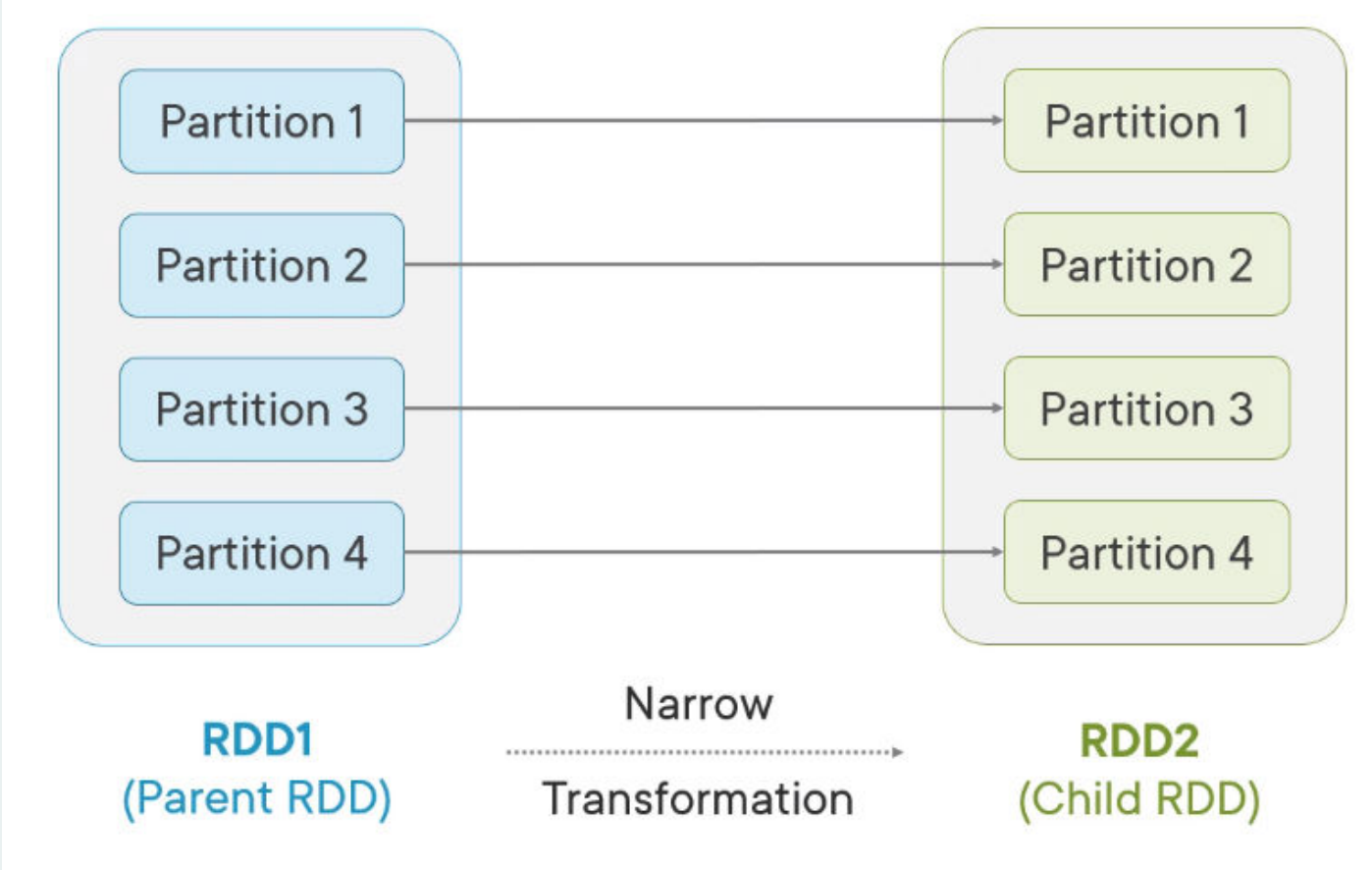
- **Narrow**
- **Wide**



Narrow transformation (1/2)

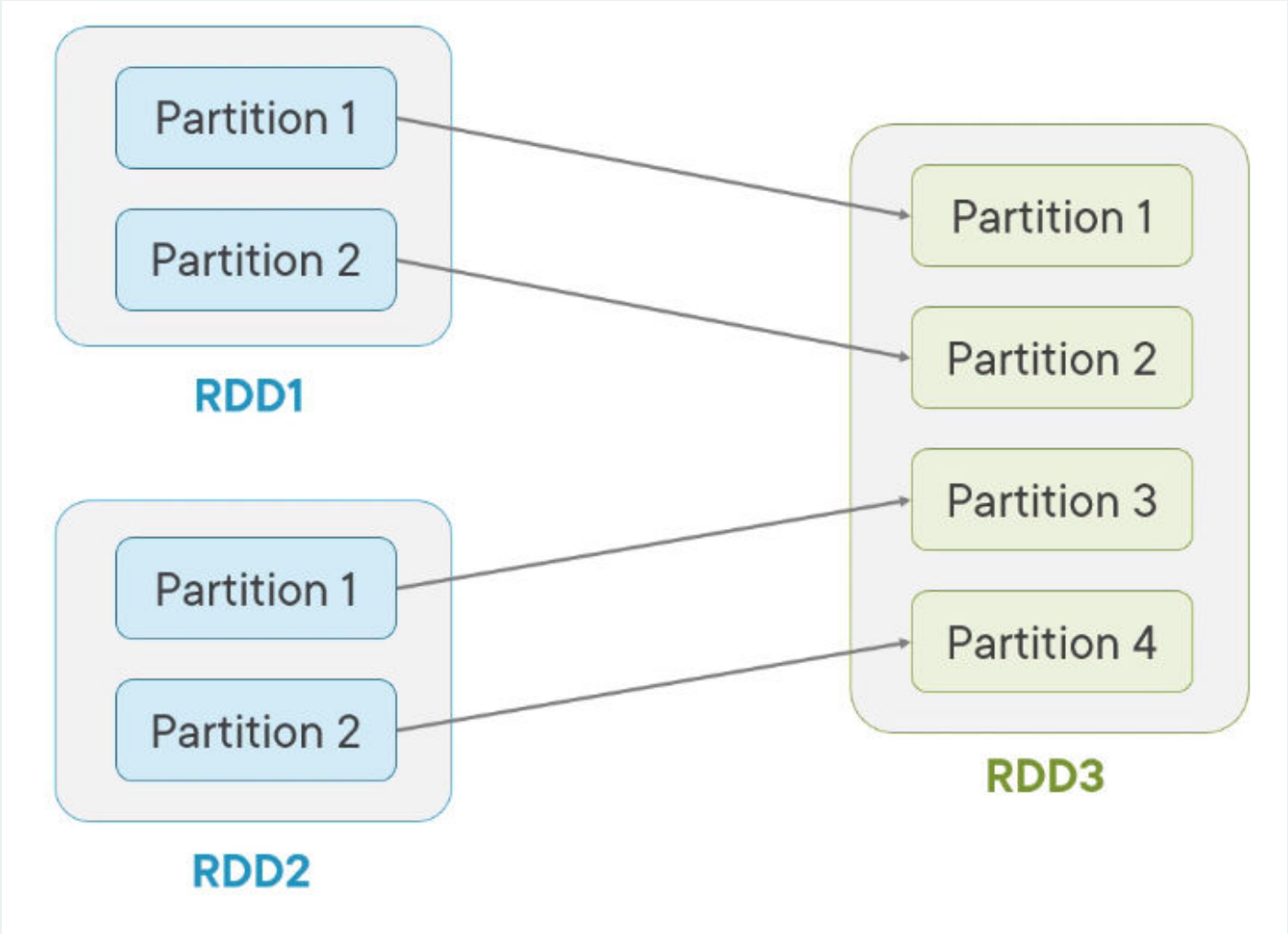
- Each input partition is used **at most once** to produce the output.
- The number of inputs is **equal** to the number of outputs.

Examples of narrow transformations are Filter, Map, MapPartition...



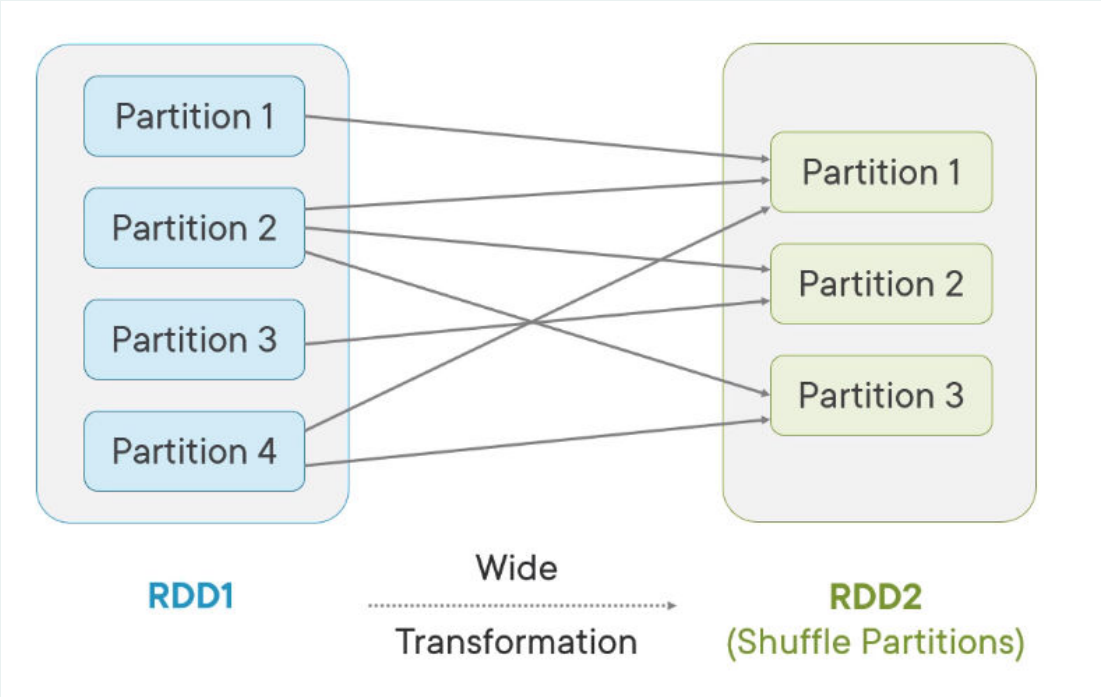
Narrow transformation (2/2)

Another correct example: Union



Wide transformation

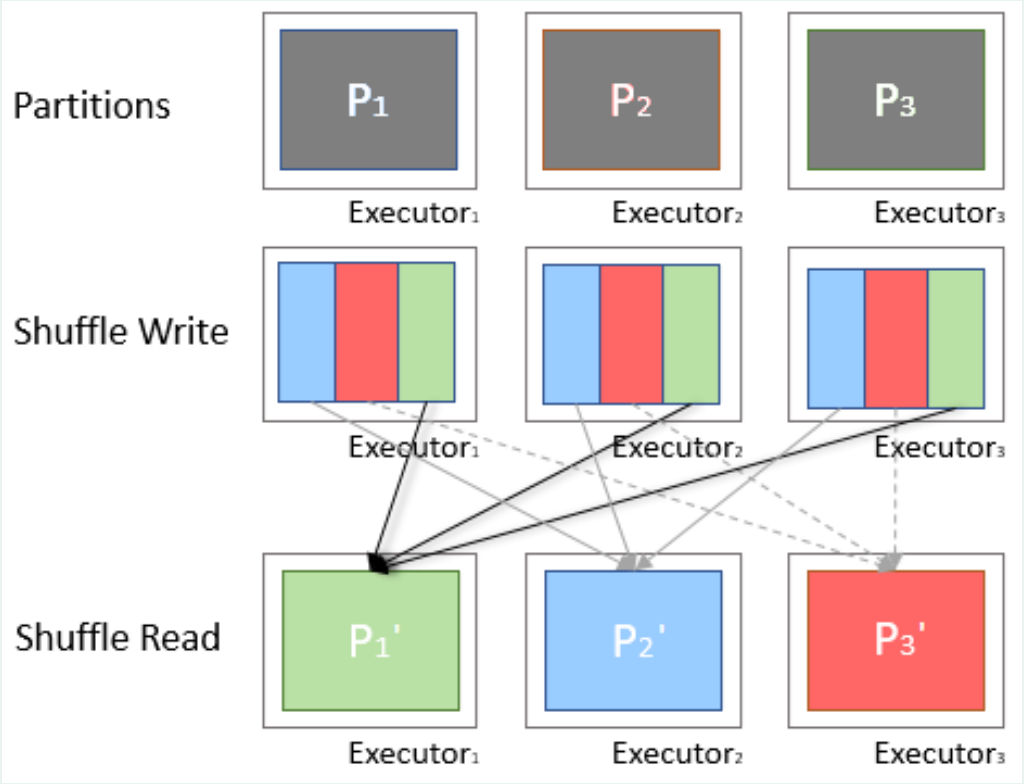
- Input partitions may be used **multiple times**.
- This induces a step of `Shuffling`.



Shuffling

- New tasks are created between steps to do the shuffle:
  - `Shuffle Write`: The Manager makes a plan on how to cut the partitions.
  - `Shuffle Read`: Executors read data from the previous shuffle write to create new partitions (their number may differ), more suitable for the task.

Note: This is an expensive operation as data must be read and written across multiple nodes



We are making good progress here!



We are now done with RDDs, let's see the Dataframes implementation in Spark.

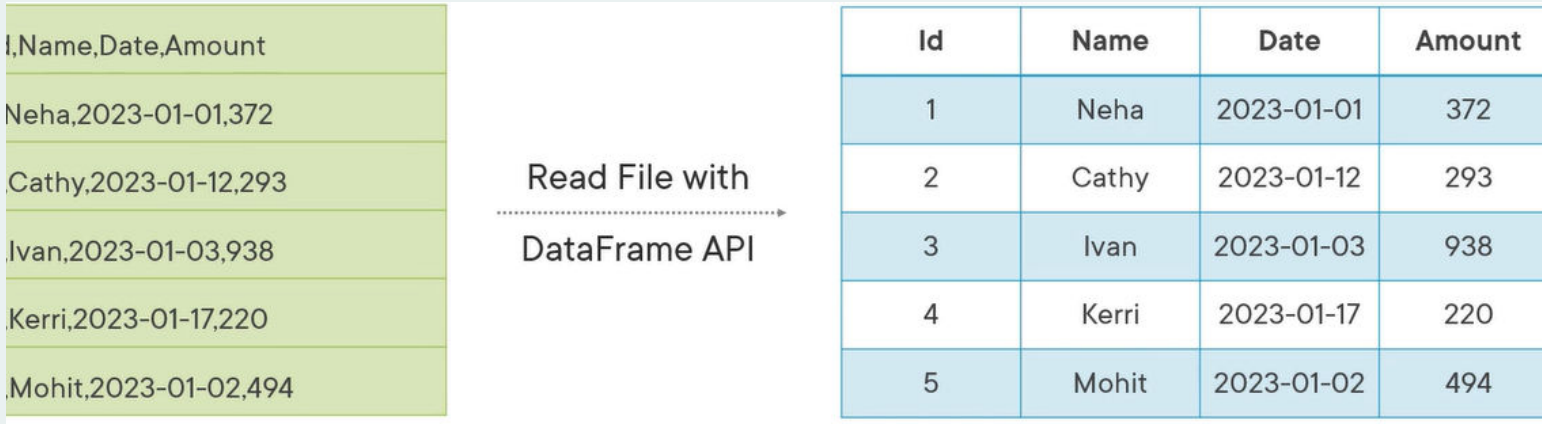
Dataframes

Like RDD, a `Dataframe` is an in-memory representation of the data in Spark. The main difference is that data is now stored in **tabular structure** (-> There are **Columns**, **rows** and each column has a **name** and **data type**).

It is built on top of RDDs and inherits most properties: **partitioned**, **read-only**, **resilient**, and **in-memory**.

It supports a lot of **types**: String, Integer, Double, Boolean, Array, Timestamp...

Spark also applies **optimizations** behind the scenes.



Create a Dataframe

Like RDDs, a Dataframe can be created following 3 ways:

- From an RDD or Collection
- Read from a data source
- Transformation of one or many other data frames

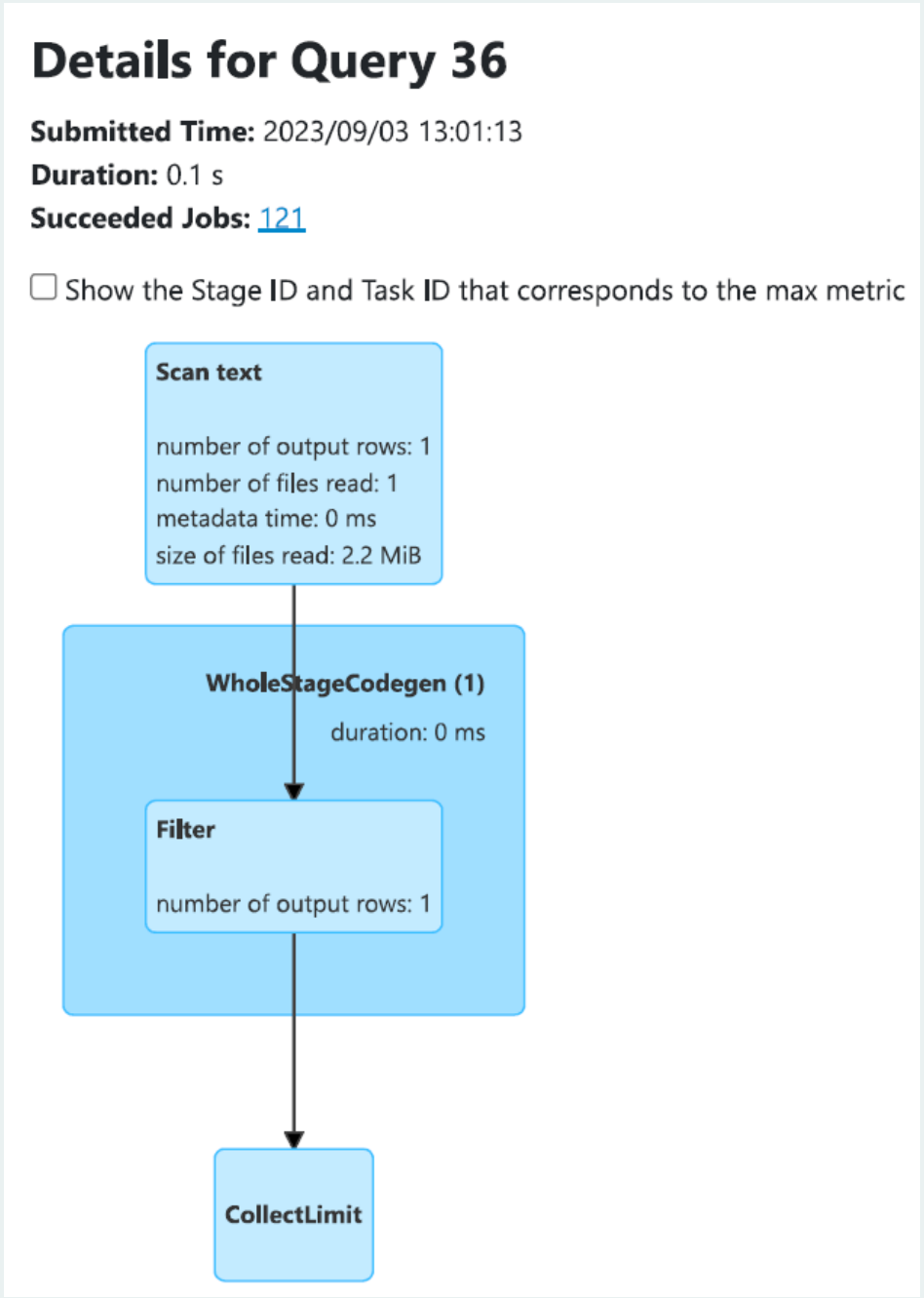
Spark Lab

- Create Dataframes & Schema (3.5)

Note on reading files through Spark (1/2)

We have seen that reading files is a `Transformation`, but if we go to the UI we can see that Spark creates a task when reading our Dataframe.

This is because Spark reads the few first bytes of a file to try to determine the number of columns in the data.



Note on reading files through Spark (2/2)

When you read files such as JSON, your schema may be more complex and nested:

```
df:  pyspark.sql.dataframe.DataFrame = [MainTag: struct]
root
 |-- MainTag: struct (nullable = true)
 |    |-- GroupId: string (nullable = true)
 |    |-- IDArray: array (nullable = true)
 |    |    |-- element: string (containsNull = true)
 |    |-- IDStruct: struct (nullable = true)
 |    |    |-- ABC-XYZ-123: array (nullable = true)
 |    |    |    |-- element: struct (containsNull = true)
 |    |    |    |    |-- BagId: string (nullable = true)
 |    |    |    |    |-- LegCorr: array (nullable = true)
 |    |    |    |    |    |-- element: struct (containsNull = true)
 |    |    |    |    |    |    |-- DelID: string (nullable = true)
 |    |    |    |    |    |    |-- SegID: string (nullable = true)
 |    |    |    |    |-- TravelerId: string (nullable = true)
 |    |    |-- BSA-ASA-211: string (nullable = true)
 |    |    |-- DSA-ASA-211: string (nullable = true)
```

Data Quality

In PySpark you have multiple checks you can apply to your data:

- **Completeness**: Remove null or missing data, fill missing values with placeholder
- **Uniqueness**: Remove duplicates
- **Accuracy**: Remove or replace inaccurate or aberrant data

Spark Lab

- Data Quality (3.6)

Optimization & performance

When you execute a transformation or an action on a Dataframe in Spark, it proceeds to some optimizations under the hood.

We have 2 optimizers:

- Catalyst Optimizer (we will see more details on it)
- Tungsten Engine (basically a memory optimizer for short, you can see more here)

Catalyst Optimizer

The Catalyst optimizer takes your queries and converts them into an execution plan.

It goes through 4 phases:

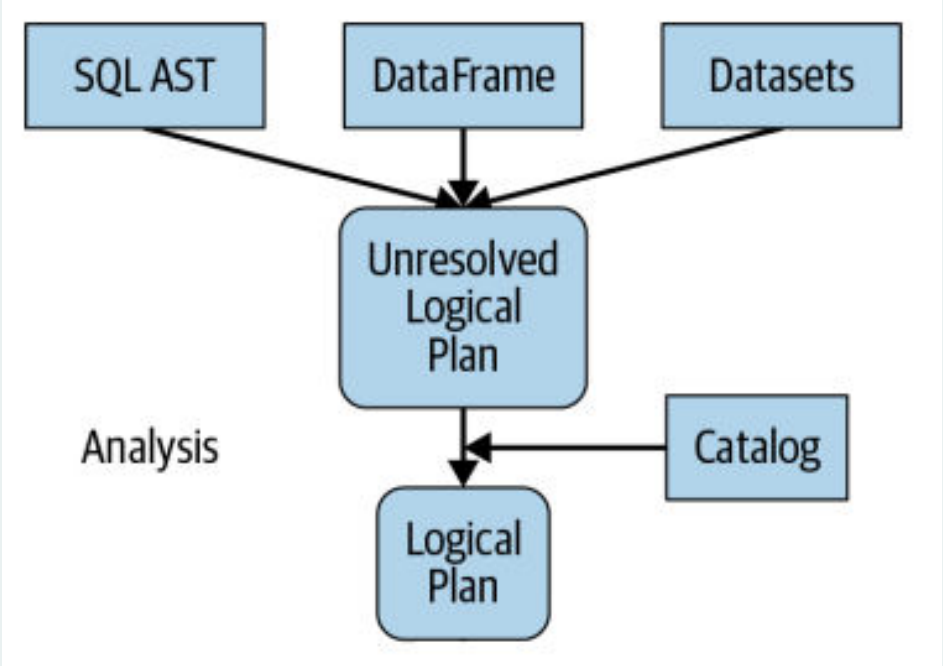
1. Analysis
2. Logical optimization
3. Physical planning
4. Code Generation

Catalyst Optimizer: Analysis

In this step, the optimizer looks at the dataframe(s) and makes sure there are no syntax errors.

It creates an `Unresolved Logical Plan` : it is "unresolved" because Spark did not check if the columns exist or were correctly declared.

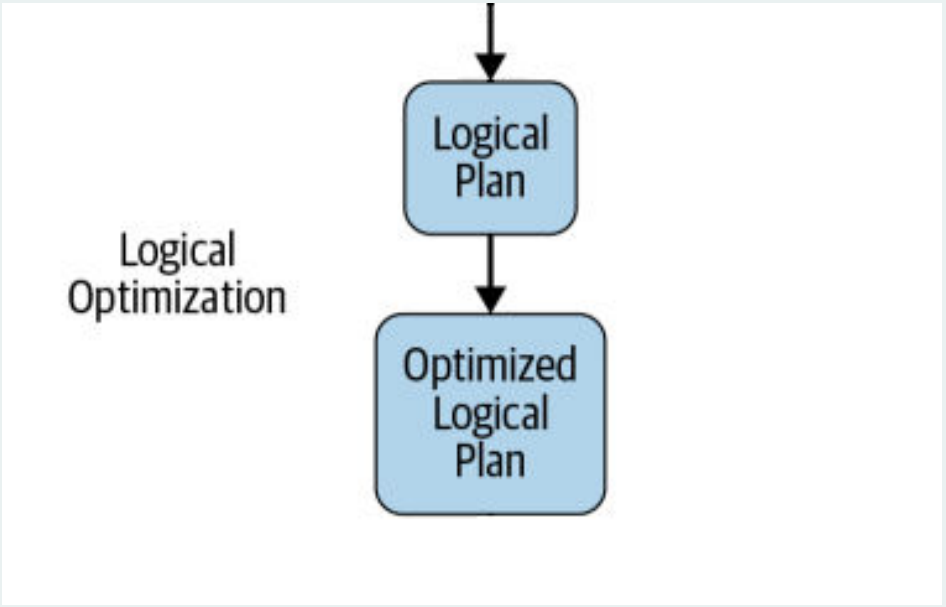
Then it checks the `Catalog` a programmatic interface to Spark that holds
a list of names of columns, data types, functions ... and validate the previous step by creating a `Logical Plan`



Catalyst Optimizer: Logical optimization

In this phase, it takes the `Logical Plan` and optimizes it following some predefined rules.

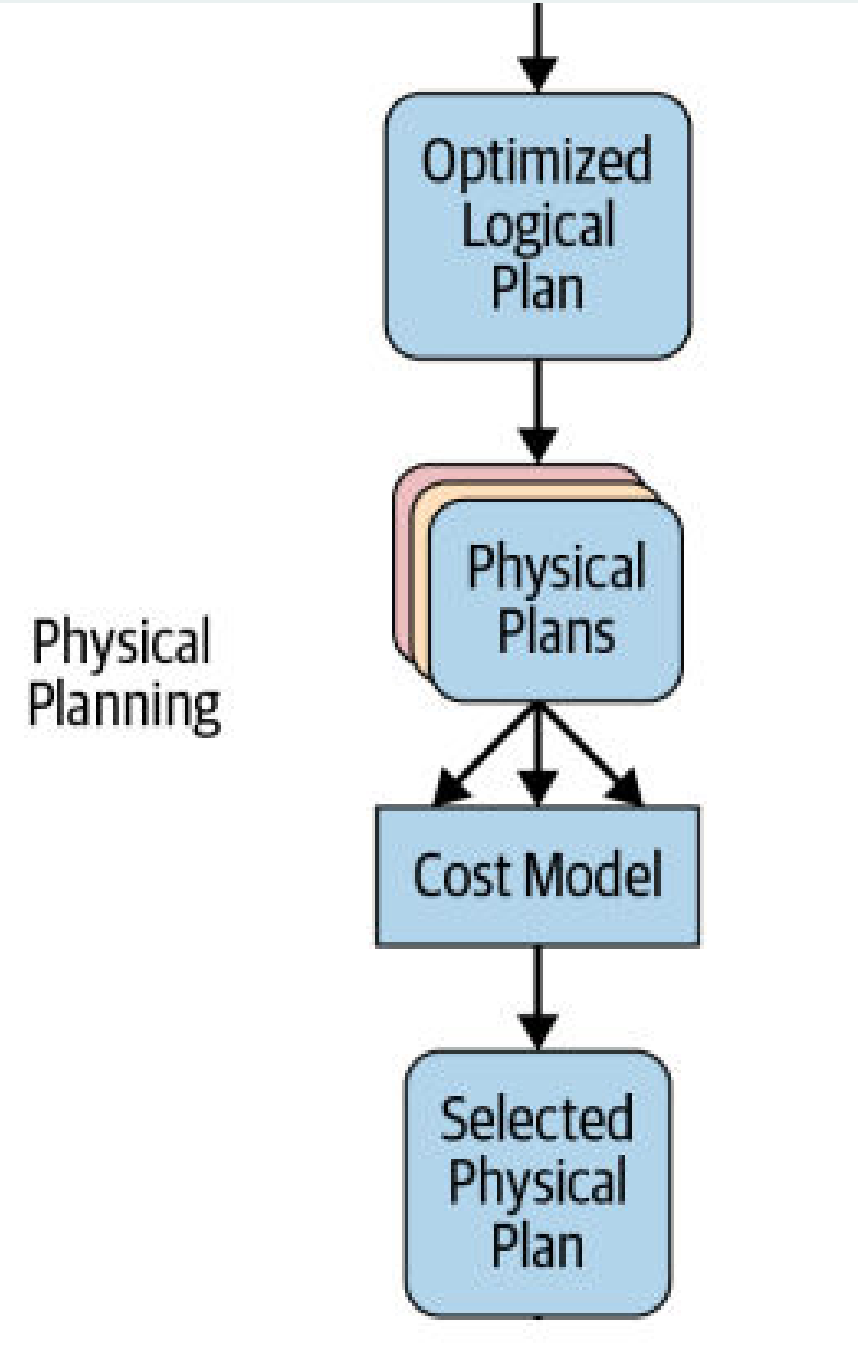
The exact optimizations are beyond the scope of this course but you can learn a bit more here.



Catalyst Optimizer: Physical planning

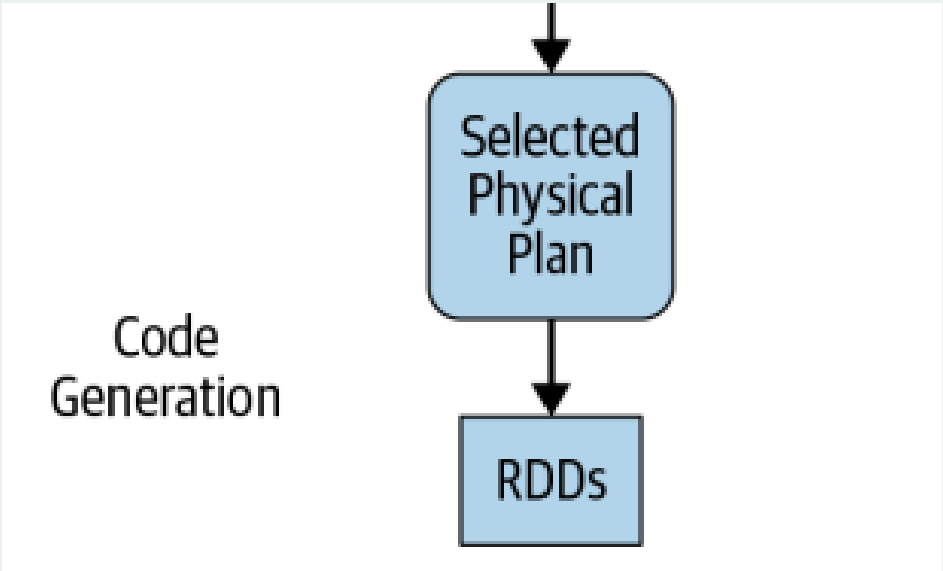
In this phase, Spark takes the selected `Optimized Logical Plan` and creates multiple `Physical Plans` based on it.

It then uses a cost-based optimizer to select the best plan and publish the final `Selected Physical Plan`.



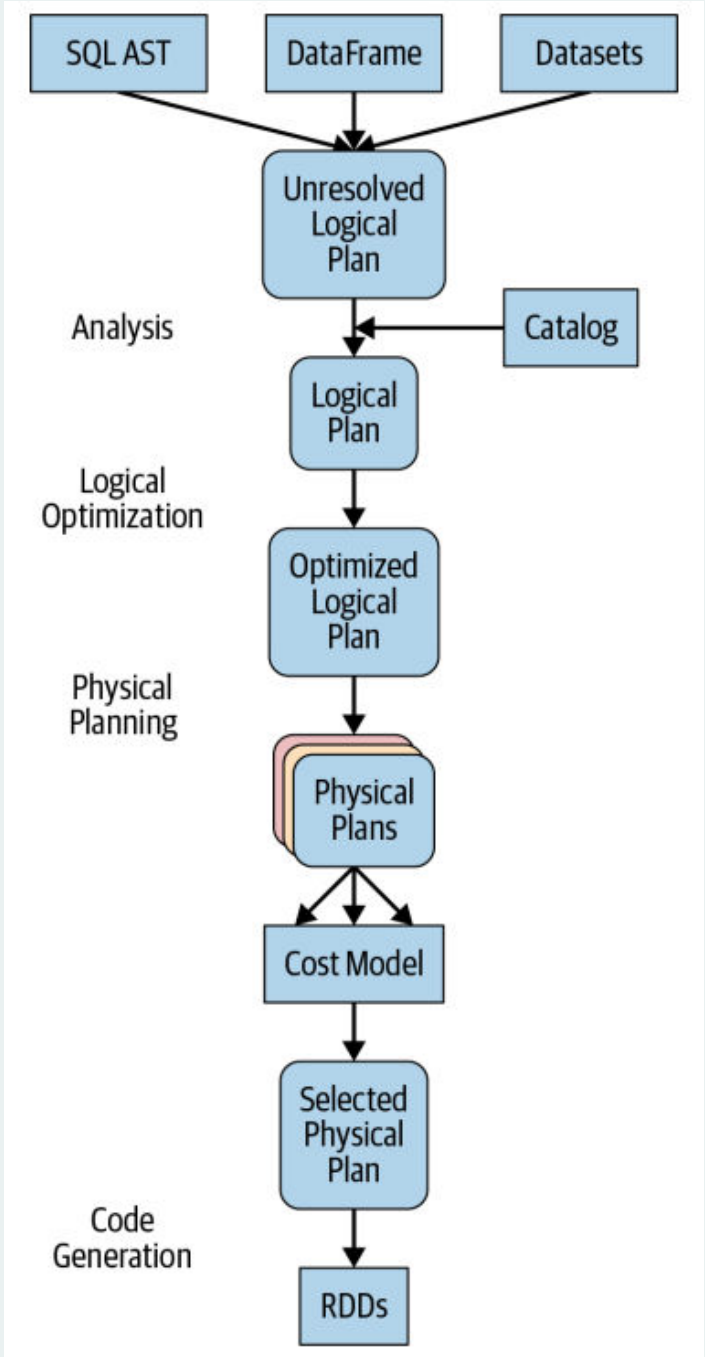
Catalyst Optimizer: Code generation

The final phase of query optimization involves generating efficient Java bytecode to run on each machine by creating `RDDs`.



Catalyst Optimizer: Complete

1. Analysis
2. Logical optimization
3. Physical planning
4. Code generation



Explain()

Note that you can see the details of each phase of the plan in Spark by using the `.explain()` function in PySpark, or in Spark UI

Details

```
== Parsed Logical Plan ==
GlobalLimit 21
+- LocalLimit 21
   +- Project [cast(v#1 as string) AS v#8, cast(count(1)#3L as string) AS count(1)#9]
      +- Aggregate [v#1], [v#1, count(1) AS count(1)#3L]
         +- SubqueryAlias `test_agg`
            +- Project [k#0, v#1]
               +- SubqueryAlias `test_agg`
                  +- LocalRelation [k#0, v#1]

== Analyzed Logical Plan ==
v: string, count(1): string
GlobalLimit 21
+- LocalLimit 21
   +- Project [cast(v#1 as string) AS v#8, cast(count(1)#3L as string) AS count(1)#9]
      +- Aggregate [v#1], [v#1, count(1) AS count(1)#3L]
         +- SubqueryAlias `test_agg`
            +- Project [k#0, v#1]
               +- SubqueryAlias `test_agg`
                  +- LocalRelation [k#0, v#1]

== Optimized Logical Plan ==
GlobalLimit 21
+- LocalLimit 21
   +- Aggregate [v#1], [cast(v#1 as string) AS v#8, cast(count(1) as string) AS count(1)#9]
      +- LocalRelation [v#1]

== Physical Plan ==
CollectLimit 21
+- *(2) HashAggregate(keys=[v#1], functions=[count(1)], output=[v#8, count(1)#9])
   +- Exchange hashpartitioning(v#1, 200)
      +- *(1) HashAggregate(keys=[v#1], functions=[partial_count(1)], output=[v#1, count#13L])
         +- LocalTableScan [v#1]
```

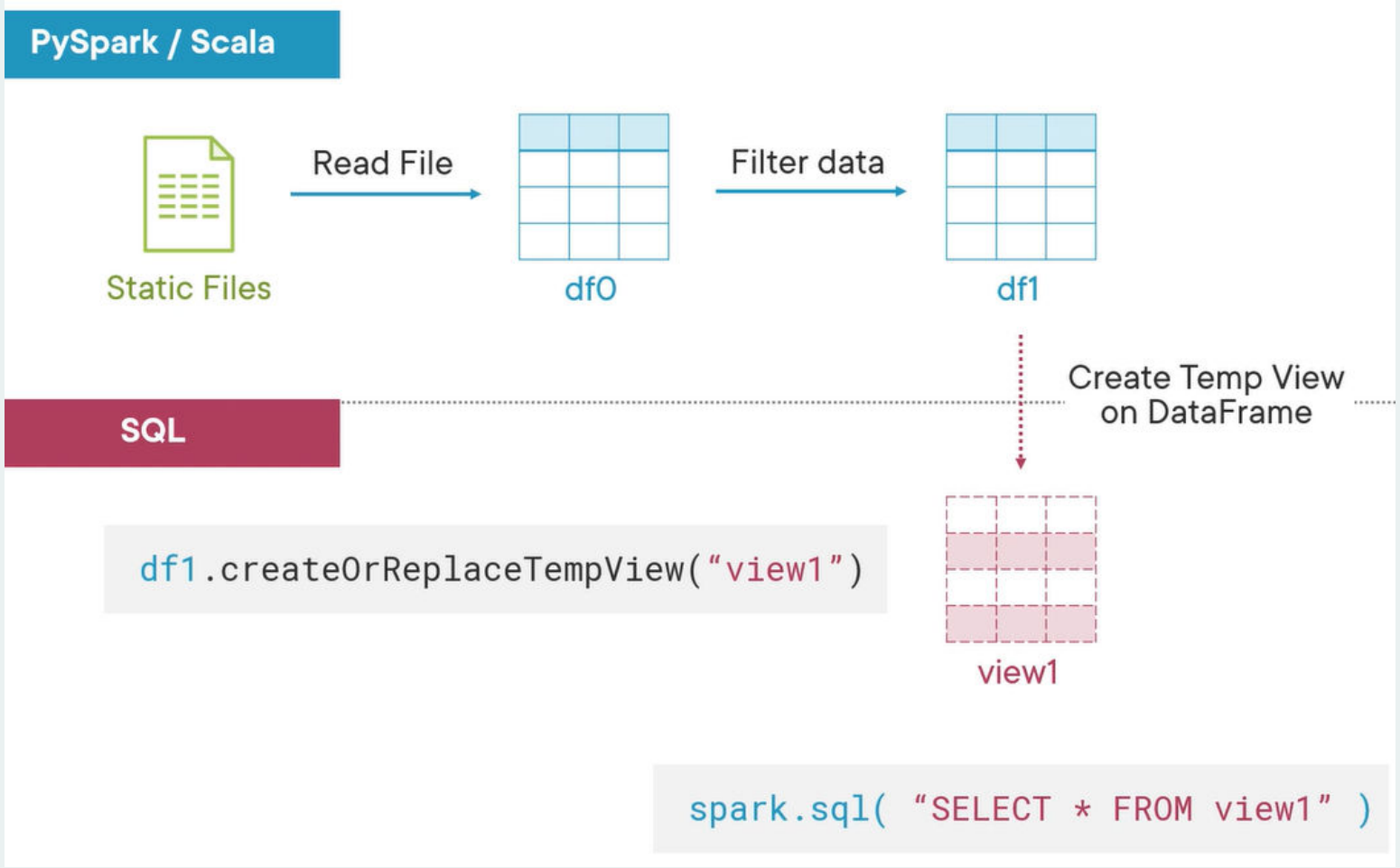
Spark Lab

- Transformations, Aggregations (3.7)

Running SQL on PySpark (1/2)

In Spark, you can use the SQL API to directly make queries in SQL.

However, you can not request on the Dataframe directly and need to create a view of it.



Running SQL on PySpark (2/2)

- The SQL view is created on top of the DataFrame, and acts as some kind of pointer (we will see the notion of catalog in a bit)
- Using SQL has the same performance as other languages thanks to the Catalyst optimizer.
- The output of the queries are a DataFrame (and not a new view for example)

Running SQL on PySpark (2/2)

SQL CHEAT SHEET http://www.sqltutorial.org

QUERYING DATA FROM A TABLE

```
SELECT c1, c2 FROM t;
```
Query data in columns c1, c2 from a table

```
SELECT * FROM t;
```
Query all rows and columns from a table

```
SELECT c1, c2 FROM t
WHERE condition;
```
Query data and filter rows with a condition

```
SELECT DISTINCT c1 FROM t
WHERE condition;
```
Query distinct rows from a table

```
SELECT c1, c2 FROM t
ORDER BY c1 ASC [DESC];
```
Sort the result set in ascending or descending order

```
SELECT c1, c2 FROM t
ORDER BY c1
LIMIT n OFFSET offset;
```
Skip *offset* of rows and return the next n rows

```
SELECT c1, aggregate(c2)
FROM t
GROUP BY c1;
```
Group rows using an aggregate function

```
SELECT c1, aggregate(c2)
FROM t
GROUP BY c1
HAVING condition;
```
Filter groups using HAVING clause

QUERYING FROM MULTIPLE TABLES

```
SELECT c1, c2
FROM t1
INNER JOIN t2 ON condition;
```
Inner join t1 and t2

```
SELECT c1, c2
FROM t1
LEFT JOIN t2 ON condition;
```
Left join t1 and t1

```
SELECT c1, c2
FROM t1
RIGHT JOIN t2 ON condition;
```
Right join t1 and t2

```
SELECT c1, c2
FROM t1
FULL OUTER JOIN t2 ON condition;
```
Perform full outer join

```
SELECT c1, c2
FROM t1
CROSS JOIN t2;
```
Produce a Cartesian product of rows in tables

```
SELECT c1, c2
FROM t1, t2;
```
Another way to perform cross join

```
SELECT c1, c2
FROM t1 A
INNER JOIN t2 B ON condition;
```
Join t1 to itself using INNER JOIN clause

USING SQL OPERATORS

```
SELECT c1, c2 FROM t1
UNION [ALL]
SELECT c1, c2 FROM t2;
```
Combine rows from two queries

```
SELECT c1, c2 FROM t1
INTERSECT
SELECT c1, c2 FROM t2;
```
Return the intersection of two queries

```
SELECT c1, c2 FROM t1
MINUS
SELECT c1, c2 FROM t2;
```
Subtract a result set from another result set

```
SELECT c1, c2 FROM t1
WHERE c1 [NOT] LIKE pattern;
```
Query rows using pattern matching %, _

```
SELECT c1, c2 FROM t
WHERE c1 [NOT] IN value_list;
```
Query rows in a list

```
SELECT c1, c2 FROM t
WHERE c1 BETWEEN low AND high;
```
Query rows between two values

```
SELECT c1, c2 FROM t
WHERE c1 IS [NOT] NULL;
```
Check if values in a table is NULL or not

Writing data

When you write data from a dataframe, you have 2 ways of doing it.

- The first one is the one we have already seen, using direct storage, for example using:

```
dataframe.write.parquet("output_path", mode="overwrite")
```

  - The file is stored in a defined format (parquet,csv...)
  - To query the file later you need to read it from storage
  - There is no metadata

Writing data

- The second option is to save it as a Spark Table:

```
dataframe.write.saveAsTable("myTable", mode="overwrite", format="parquet")
```

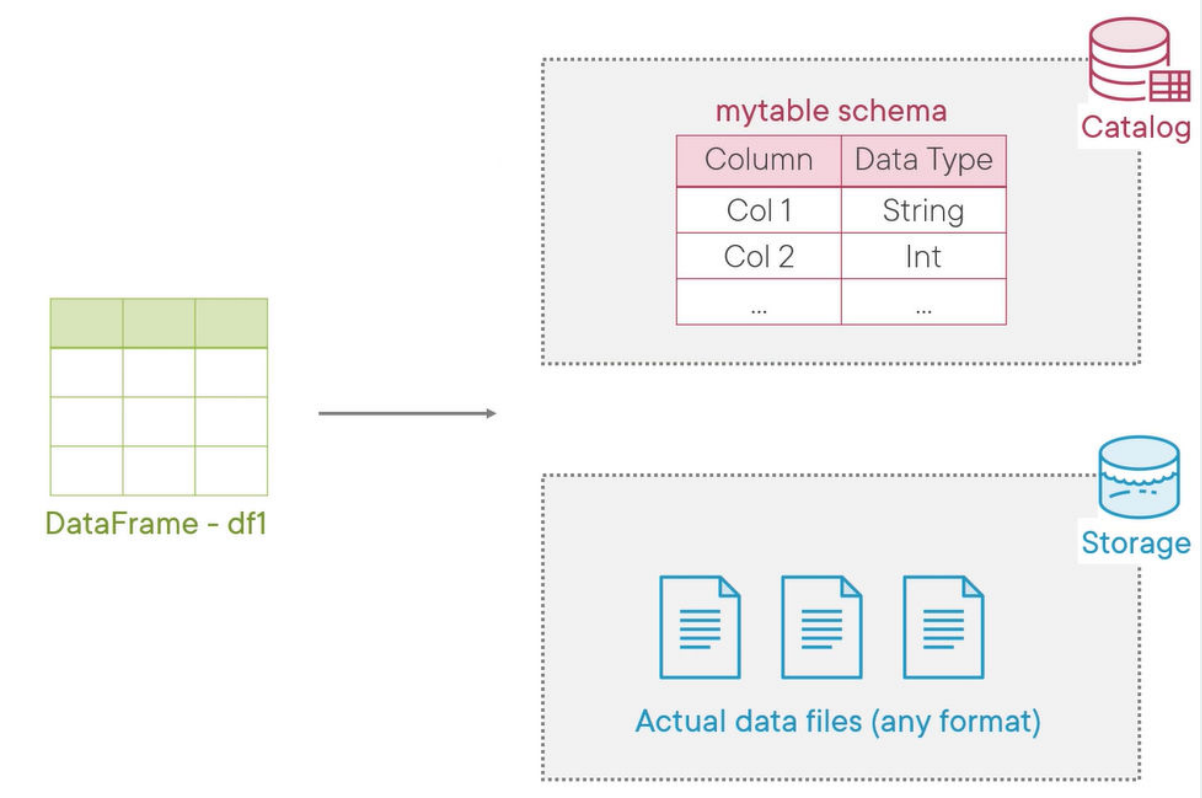
  - When you save a DataFrame as a table, Spark does not create a local file directly in your working directory but in a table to a persistent catalog
  - A metastore is created registering metadata
  - To query the file later you can directly use a reference to the table instead of reading the file again.

When you want to read your file

You can do it by calling the table directly from the catalog:

```
Select * From mytable
```

```
df1 = spark.read.table("mytable")
```



Spark Lab

- Spark SQL (3.8)

Catalogs

There are 2 types of catalogs:

- **In-memory Catalog**
  - Work for the current session
  - It is cleaned up when it ends and nothing is persisted
- **Persistent Catalog**
  - Metadata is permanently stored on the cluster
  - You can use it later in a new sessions
  - Spark has a built-in catalog (generally Hive, but can be another tech such as Hadoop).

Spark Tables

You have 2 types of Spark Tables as well:

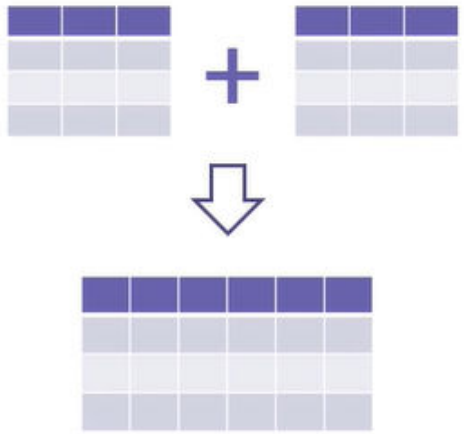
- **Managed Table**
  - Schema and data are managed by Spark internally.
  - Data is located in memory.
  - If you drop a Managed table, both Data and schema are lost.
  - The main advantage of this is to persist staging data.
- **Unmanaged** or **External table**
  - only the schema is managed by Spark internally.
  - Data is stored in an external storage.
  - If you drop an Unmanaged table, only the schema is lost as data is only referenced in the table and stored elsewhere
  - Useful to persist the data you processed

Spark Lab

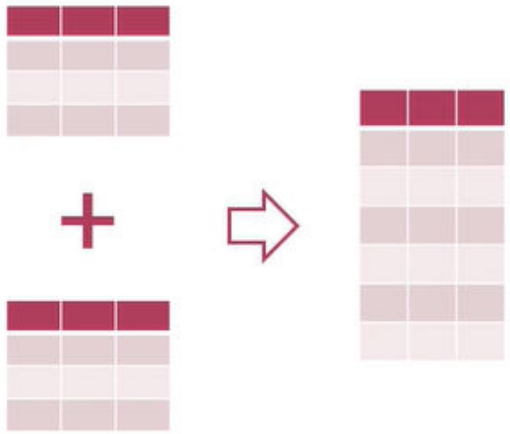
- Spark Tables (3.9)

Work with multiple Dataframes

We will see how to interact with multiple Dataframes, mainly with Join operations and set operations.



Join Operations

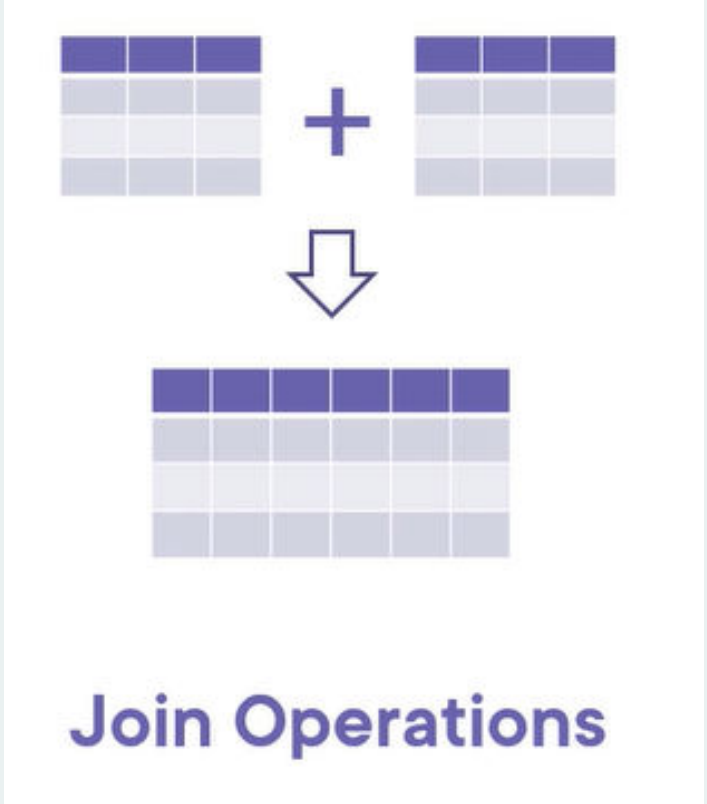


Set Operations

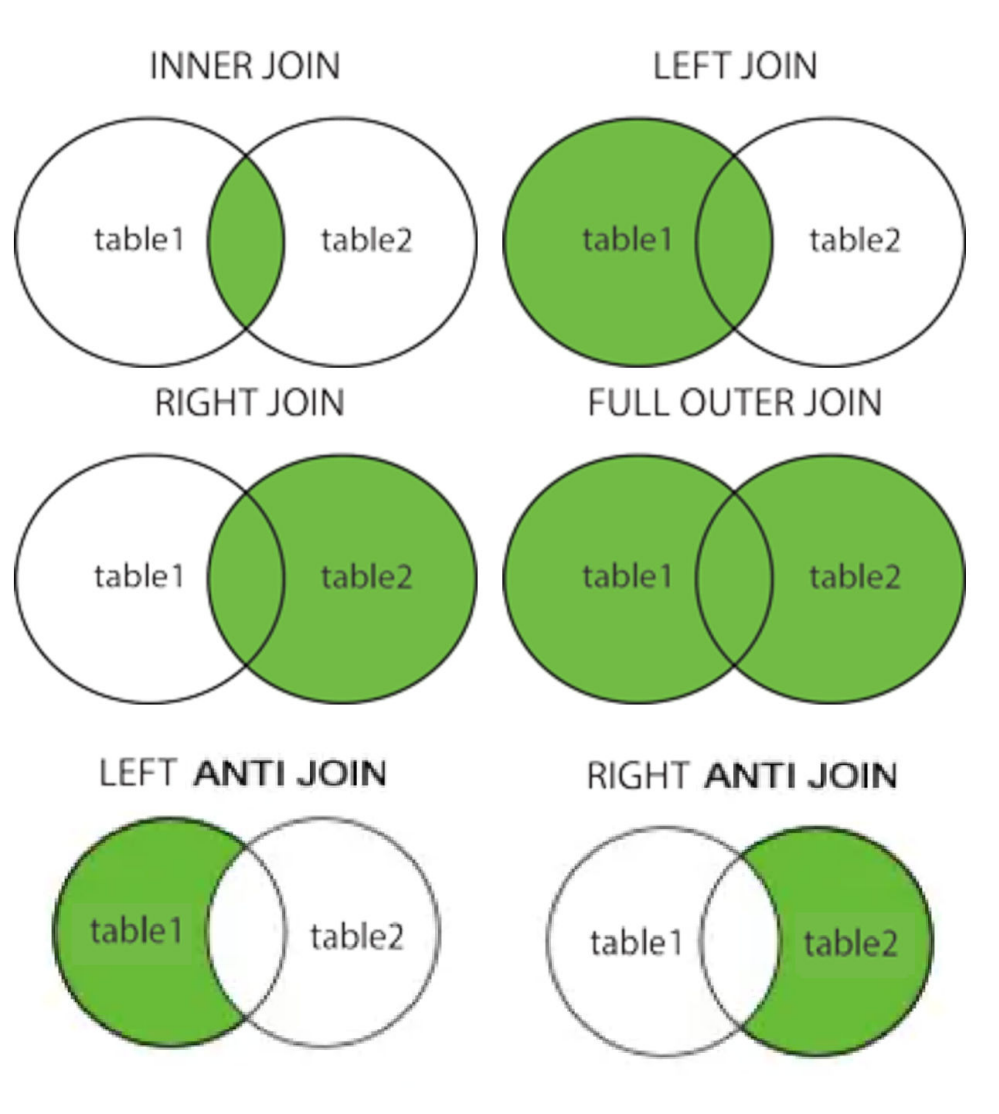
Join Operations

Spark supports the following kind of joins:

- Inner Join
- Left (outer) Join
- Right (outer) Join
- Full (outer) Join
- Cross Join *(only in python & Scala)*
- Semi Join *(only in python & Scala)*
- Anti Join *(only in python & Scala)*



Most common Joins



Inner Join

Table1:

id	name
1	Alice
2	Bob

Table2:

id	age
1	25
3	30

```
SELECT Table1.id, Table1.name, Table2.age
FROM Table1
INNER JOIN Table2 ON Table1.id = Table2.id
```

id	name	age
1	Alice	25

Left Outer Join (Same idea for Right Outer Join but in reverse)

Table1:

id	name
1	Alice
2	Bob

Table2:

id	age
1	25
3	30

```
SELECT Table1.id, Table1.name, Table2.age
FROM Table1
LEFT OUTER JOIN Table2 ON Table1.id = Table2.id
```

id	name	age
1	Alice	25
2	Bob	NULL

Full Outer Join

Table1:

id	name
1	Alice
2	Bob

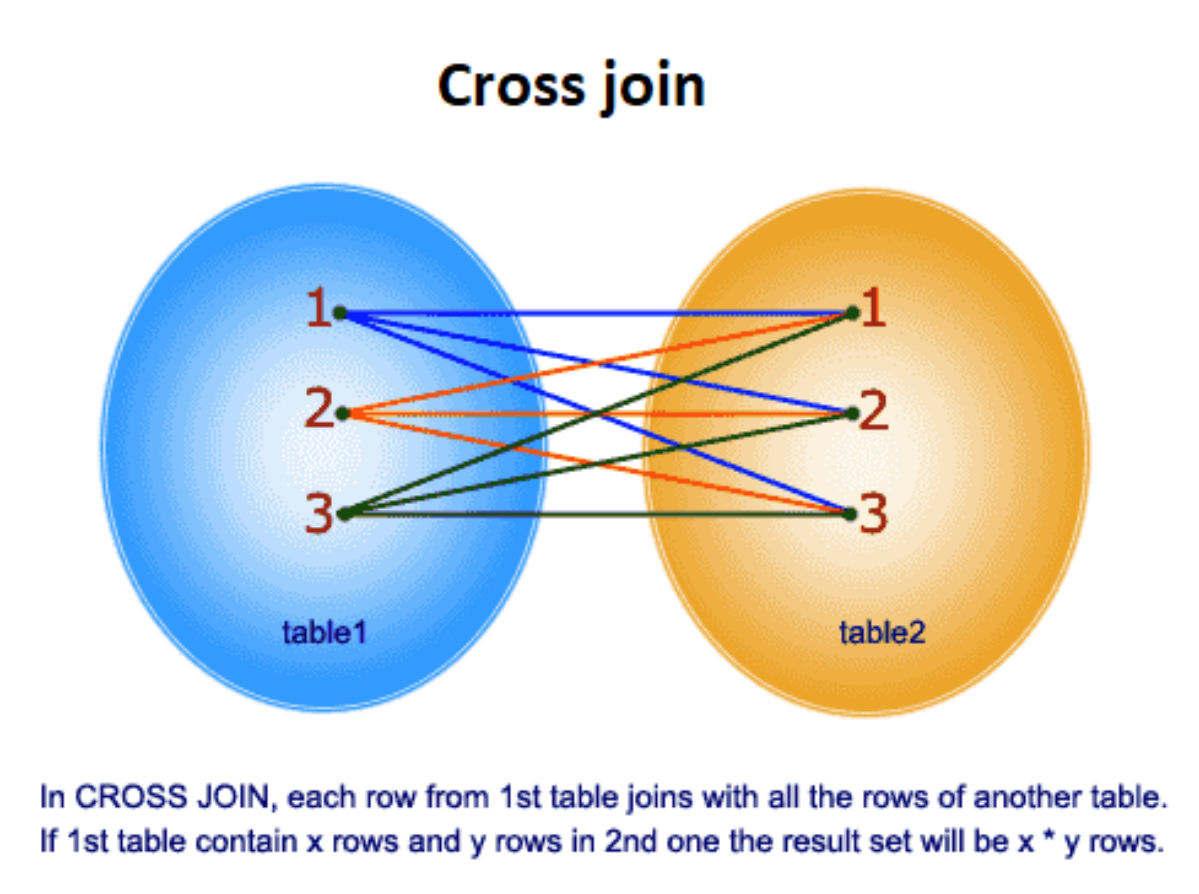
Table2:

id	age
1	25
3	30

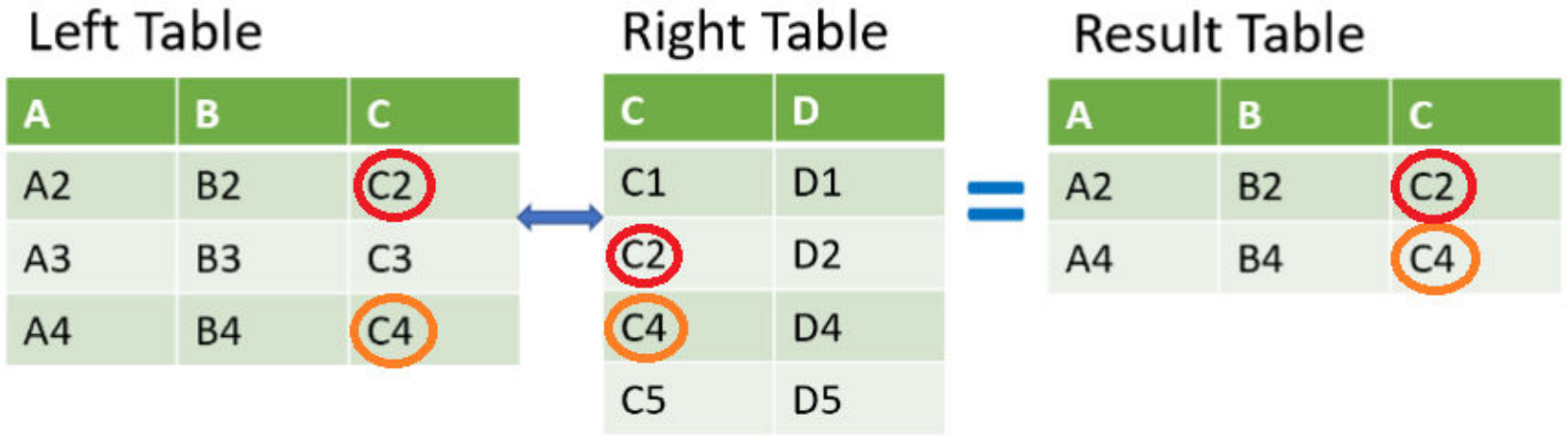
```
SELECT COALESCE(Table1.id, Table2.id) AS id, Table1.name,
Table2.age
FROM Table1
FULL OUTER JOIN Table2 ON Table1.id = Table2.id
```

id	name	age
1	Alice	25
2	Bob	NULL
3	NULL	30

Cross Join


Cross join
1
2
3
1
2
3
table1
table2
In CROSS JOIN, each row from 1st table joins with all the rows of another table.
If 1st table contain x rows and y rows in 2nd one the result set will be x * y rows.

What is a semi-join?



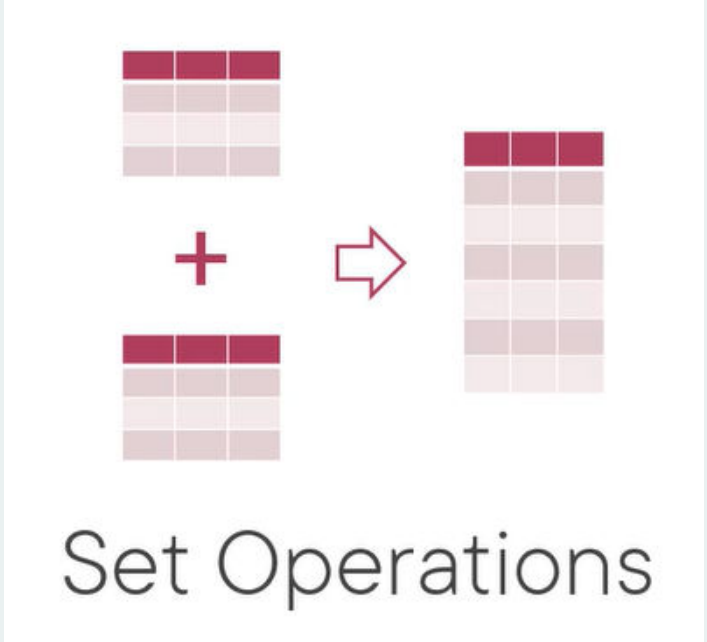
Semi-joins

- Returns the intersection, similar to an inner join
- Returns only columns from the left table and ***not*** the right
- No duplicates

Set Operations

Spark supports the following kind of sets:

- Union
- Union All
- Intersect
- Except/Minus



Union

Table1:

id	name
1	Alice
2	Bob

Table2:

id	name
1	Alice
3	Carol

```
SELECT * FROM Table1
UNION
SELECT * FROM Table2
```

id	name
1	Alice
2	Bob
3	Carol

`UNION` returns unique rows from both tables, eliminating duplicates.

Union All

Table1:

id	name
1	Alice
2	Bob

Table2:

id	name
1	Alice
3	Carol

```
SELECT * FROM Table1
UNION ALL
SELECT * FROM Table2
```

Result:

id	name
1	Alice
2	Bob
1	Alice
3	Carol

`UNION ALL` returns all rows from both tables, including duplicates.

Intersect

Table1:

id	name
1	Alice
2	Bob

Table2:

id	name
1	Alice
3	Carol

```
SELECT * FROM Table1
INTERSECT
SELECT * FROM Table2
```

id	name
1	Alice

Except / Minus

Table1:

id	name
1	Alice
2	Bob

Table2:

id	name
1	Alice
3	Carol

```
SELECT * FROM Table1
EXCEPT
SELECT * FROM Table2
```

id	name
2	Bob

Spark Lab

- Work with multiple Datasets (3.10)

UDFs

Sometimes you will want to do something that is not covered by existing functions and APIs.

You can extend the PySpark capabilities by creating custom functions called `User Defined Functions` or UDF for short.

- A UDF will be invoked for each row in your Dataframe
- You can register it using `udf()` for Dataframes
- You can register it using `spark.udf.register()` for SQL

Caution!

UDFs often come as a last resort.

If you find a function that already does what you need use it:

- You will gain time while developing.
- PySpark will never apply optimizations as it is a custom function.
- Nulls or any other edge case or exception need to be handled properly by you
- There is a good chance that the function you use coming from a known library will be maintained and updated across different versions.

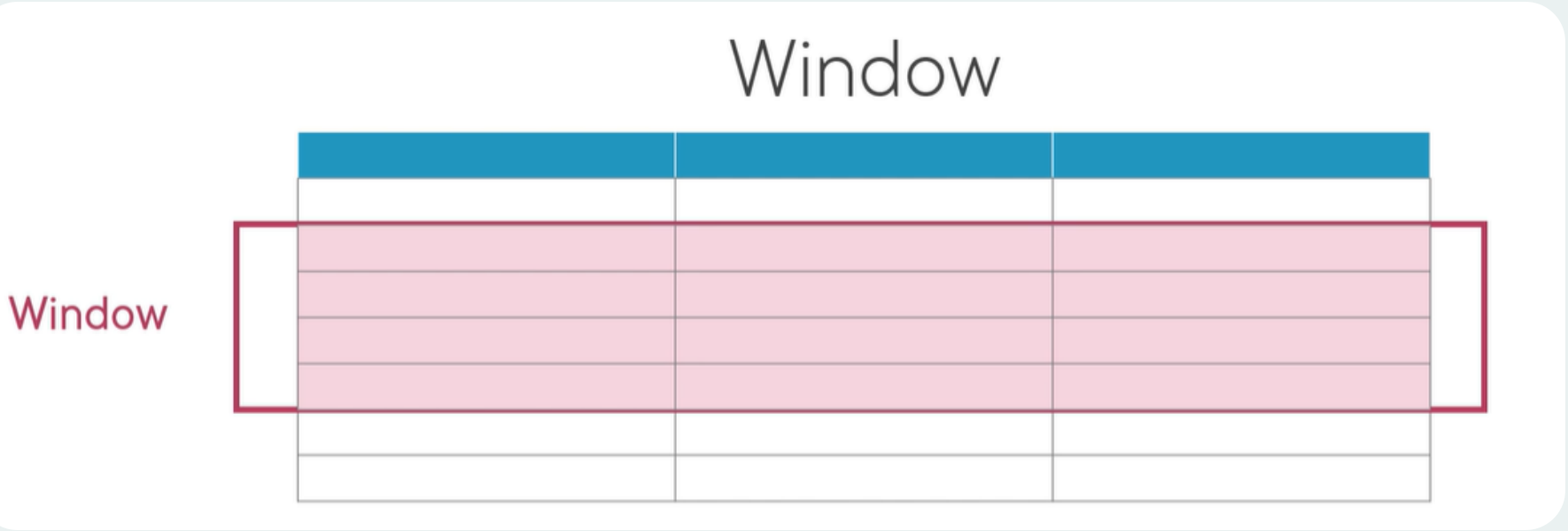
This is very important in a production environment where you will already have to keep track of a lot of evolutions and dependencies in your projects.

Windows Functions

A Window function is a subset of related rows in a DataFrame.

For example, they can be related by time, location, ...

This will allow us to apply operations on all rows of a Window.



Simple example

Let's take an example of expenses in 4 departments.

In this case, our window will contain all 4 rows and we want to calculate the percentage of expenses of each unit.

Department	Expense
HR	10,000
Tech	20,000
Sales	30,000
Marketing	40,000

Simple example

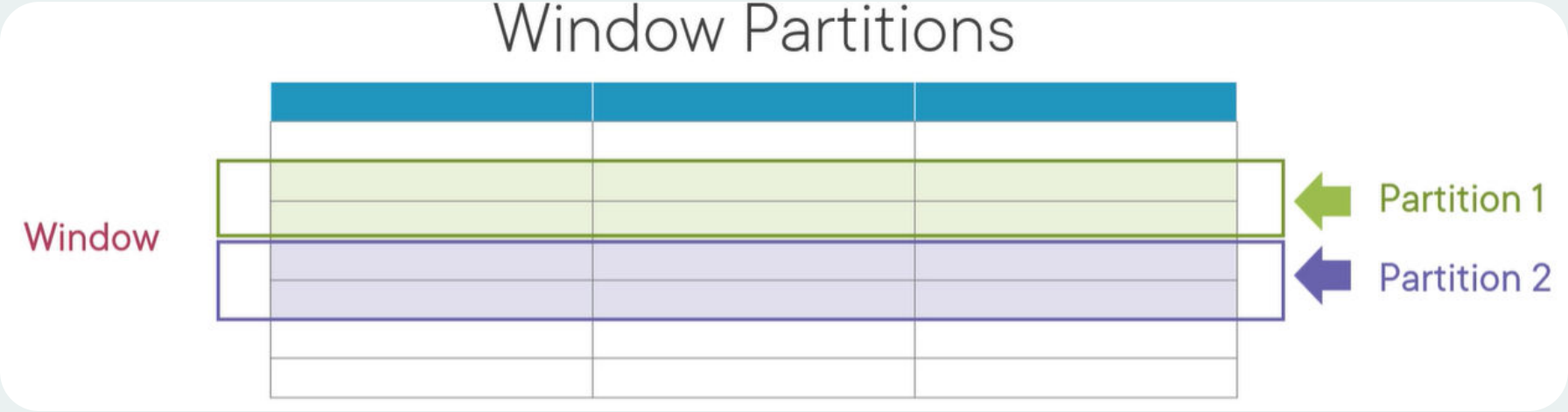
Department	Expense	Total Expenses	Expense %
HR	10,000	100,000	10
Tech	20,000	100,000	20
Sales	30,000	100,000	30
Marketing	40,000	100,000	40

```
SELECT *
        , Expense * 100 / TotalExpenses          AS ExpensePercent
FROM (
            SELECT Department, Expense
                        , SUM (Expense)  OVER ()     AS TotalExpenses
            FROM Departments
                                                  Window function on the
                                                  whole dataset
        ) subquery
```

Partitions

Things get interesting when you split your Window into smaller partitions:

Each partition may contain a different number of rows.



Partition example

We want to find the share of employee payout for its department.

Our Window is the entire dataset, and our partitions are each department.

Department	Employee	Salary
Sales	Anna	8,000
Marketing	Steve	9,500
Tech	Cristina	9,000
Marketing	Neha	10,500
Tech	Kari	10,000
Sales	Ivan	10,000
Tech	Mohit	8,000

Partition example

Department	Employee	Salary	Dept Payout	Share %
Sales	Anna	8,000	18,000	44.4
Sales	Ivan	10,000	18,000	55.6
Marketing	Steve	9,500	20,000	47.5
Marketing	Neha	10,500	20,000	52.5
Tech	Cristina	9,000	27,000	33.3
Tech	Kari	10,000	27,000	37.0
Tech	Mohit	8,000	27,000	26.7

```
SELECT *
      , Salary * 100 / DeptPayout                                  AS SharePercent
FROM (
        SELECT Department, Employee, Salary
              , SUM (Salary)  OVER ( PARTITION BY Department )  AS DeptPayout
        FROM Departments
      ) subquery
```

Window Partitions

Spark Lab

- Advanced functions (3.11)

Partitions

Partitioning is an important aspect of Spark that will help you optimize the time and resources allocated to a specific task.

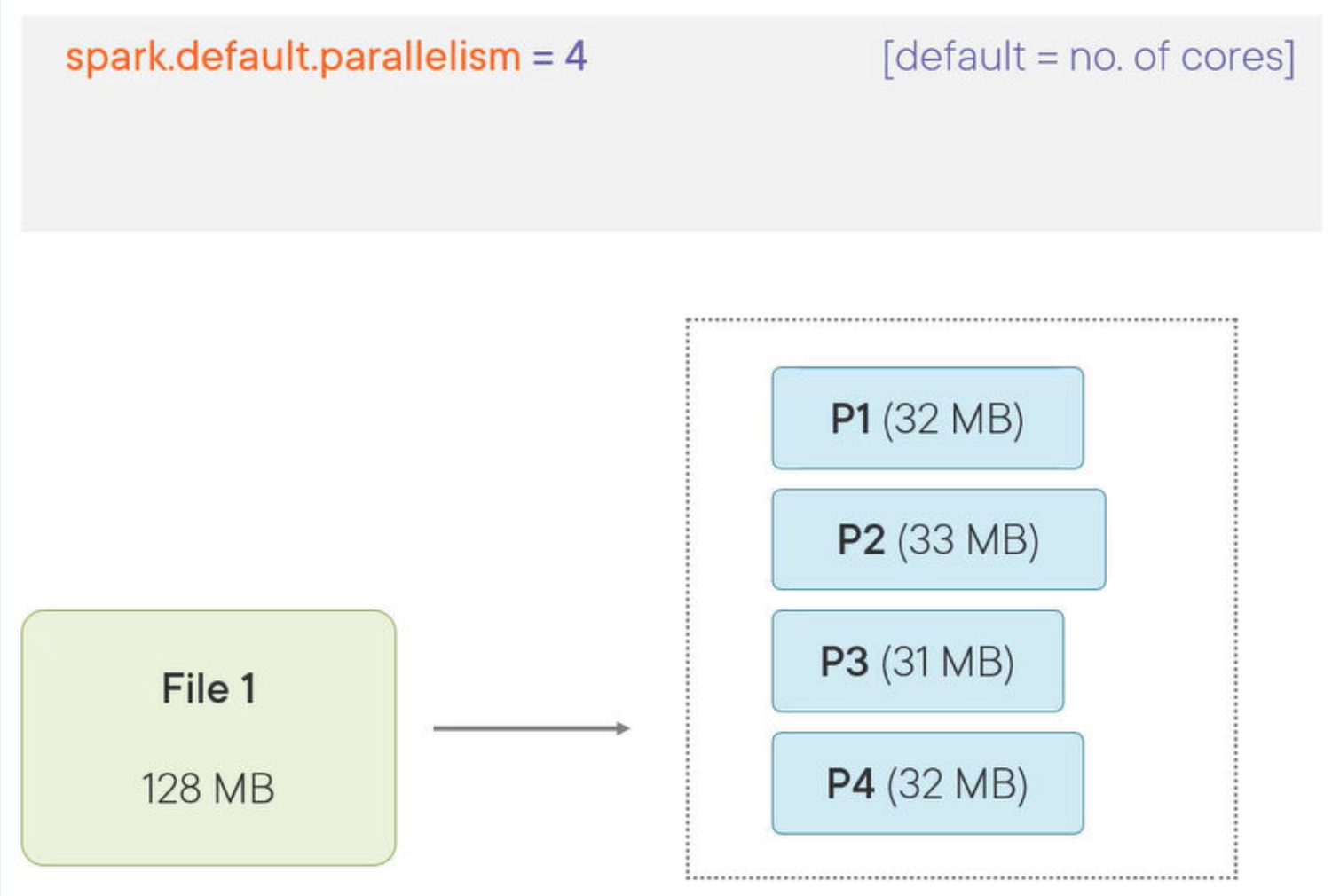
Two types of partitioning

- **In-memory**
  - Chunks of data stored in the RAM
  - All chunks combined constitute either an RDD or a Dataframe in PySpark
- **Disk**
  - Data is physically saved on a disk, partitioned based on columns

Both help parallelize jobs.

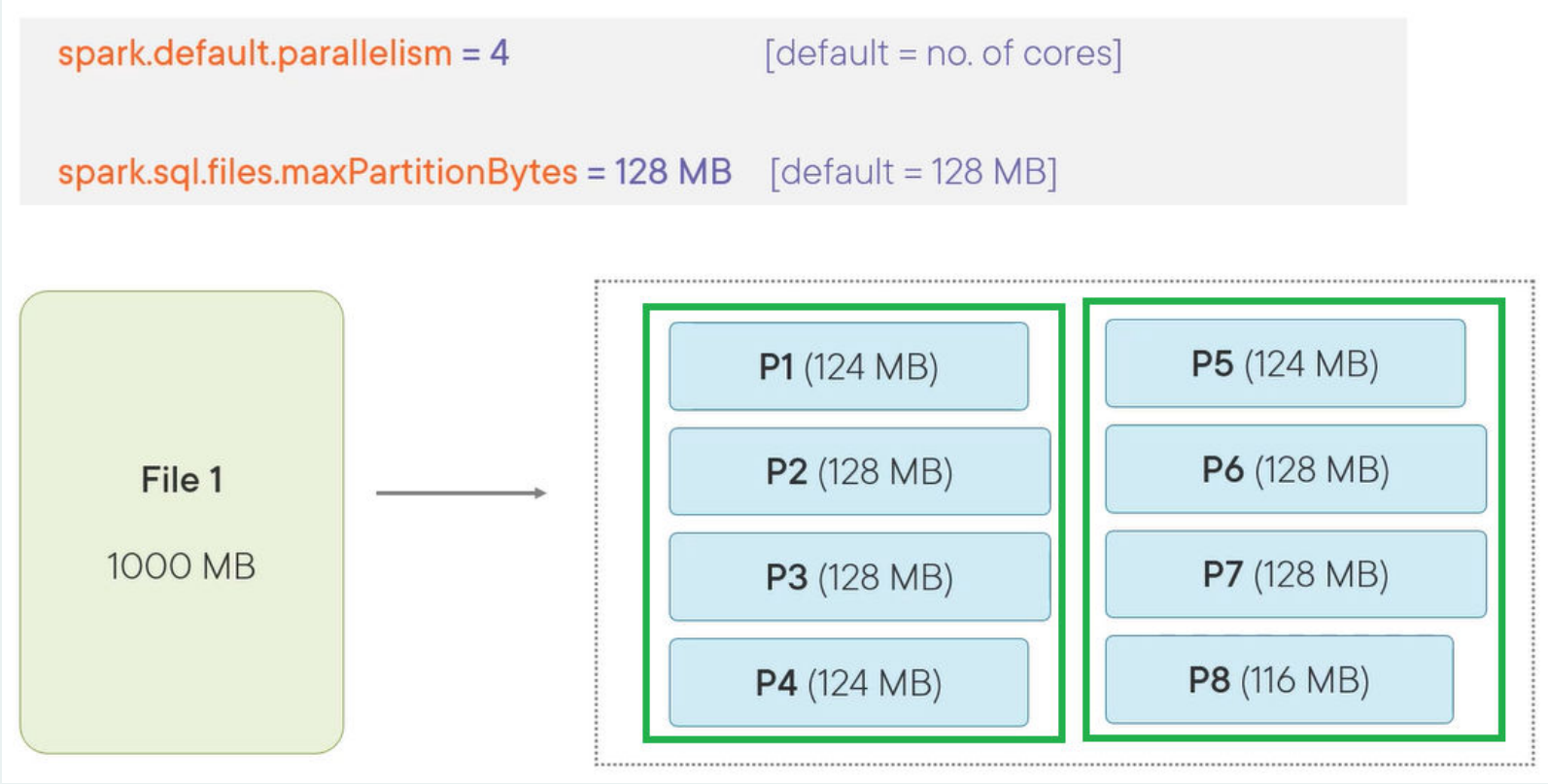
Partitioning: Reading data (1/2)

- The `parallelism` setting will define the target number of cores to use in parallel
- Splits will be made roughly equal to one another
- Spark may create fewer partitions if there is not enough data to split



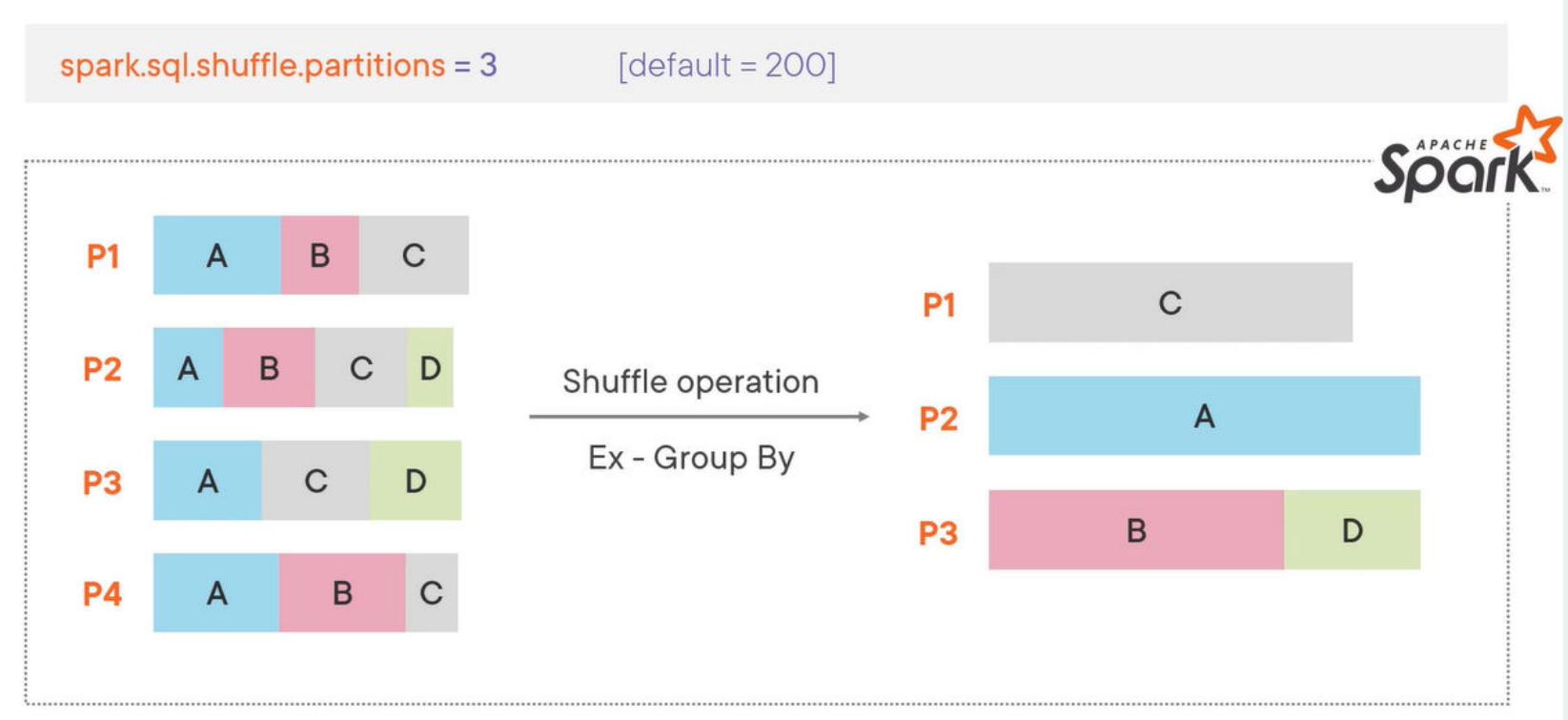
Partitioning: Reading data (2/2)

- Another setting `maxPartitionBytes` will define the maximum size of a partition
- Spark will do its best to follow both parameters but may not be able to



Partitioning: Shuffle

- `shuffle.partitions` will define the target number of partitions to create in a shuffle operation



Partition Trade-off

As a reminder:

Having very few/big partitions:

- Each task may need to process lot of data
- Cluster resources may be under-utilized

Having lot of/small partitions:

- Too many tasks are created
- Reduces parallelism since tasks go in waiting state

Two methods to change partitions (1/2)

Repartition

- Typically used to **increase** the number of partitions (because it is a **wide transformation** inducing shuffling)
- Often useful to rebalance data when partitions are not of the same size naturally or to reduce the size of existing partitions.
- Different options:
    - Round Robin (equal-sized partitions, regardless of content)
    - Hash (co-locate data based on columns, for example, same country)
    - Range (sort **&** co-locate)

Two methods to change partitions (2/2)

Coalesce

- Only used to **decrease** the number of partitions
- Is a **narrow transformation** (no shuffle)
- You have to be cautious, as if your new partitions are too large it may result in memory exceptions
- Used when partitions are too small or the output requires a reduced number of files

Spark Lab

- Partitioning (3.12)

We are done with the Spark course!



To go further on your own:

- Free book Learning Spark (Second Edition)
- Databricks Community Edition with some free tutorials and resources
- Last year's courses

Assignement

- Assignement directives (3.20)

Credit and used resources:

- (Not Free) Pluralsight: Apache Spark 3 Fundamentals by Mohit Batra
- (Not Free) Pluralsight: Apache Spark Fundamentals by Justin Pihony
- Hadoop & Spark courses from previous years by Olivier Moïse (2022) & Fabrice Woittequand (2020)
- Dataflair Spark APIs
- Actions, Narrow Transformations, and Wide Transformations by Chen Ziyu
- Learning Spark (Second Edition)